CodeGuard

Software Supply Chain Risk Simulator — Complete Technical Documentation

Application	Django 5.2 · Python 3.12 · SQLite · Matplotlib
Source	github.com/bsclmr111626/software-engineering-project, commit 8ca8b36 (CI passing)
Author	bsclmr134322-collab
Generated	10 November 2026, from the Markdown files in docs/complete/
Contents	Master document, eight detailed documents, eight diagrams, six reference inventories

Contents

- **Part I — Master document**
  - CodeGuard — Complete Technical Documentation
    1. 1. Purpose
    2. 2. Technology stack
    3. 3. Directory structure
    4. 4. Backend
    5. 5. Frontend
    6. 6. Database
    7. 7. Authentication, authorization and security
    8. 8. Threat modelling in CodeGuard
    9. 9. Simulation and risk scoring
    10. 10. Controls and recommendations
    11. 11. Remediation
    12. 12. Reporting, charts and analytics
    13. 13. Audit trail
    14. 14. Testing and quality assurance
    15. 15. Git, GitHub, Jenkins and CI/CD
    16. 16. Configuration and environment variables
    17. 17. Installation and deployment
    18. 18. Performance
    19. 19. Maintainability
    20. 20. Troubleshooting (most common)
    21. 21. Limitations and future work
    22. 22. Glossary and learning guide
    23. Appendix — How this documentation was verified
- **Part II — Detailed documents**
  - Architecture
  - Database and migrations
  - Security
  - Simulation and risk methodology
  - Testing and quality assurance
  - Development and deployment
  - Learning guide
  - Glossary
- **Appendix A — Diagrams**
  - Diagram: system architecture
  - Diagram: request lifecycle
  - Diagram: data flow
  - Diagram: authentication and authorization
  - Diagram: simulation workflow (form to report)
  - Diagram: propagation algorithm
  - Diagram: entity-relationship model

    - Diagram: CI/CD
- **Appendix B — Reference inventories**
    - Reference: models and database tables
    - Reference: URL routes
    - Reference: migrations
    - Reference: dependencies
    - Reference: environment variables
    - Reference: automated tests

Part I — Master document

CodeGuard — Complete Technical Documentation

Software Supply Chain Risk Simulator · documentation generated from the source at commit `8f5d092` plus the working tree, on 2026-11-10; updated after commits `eef6716` (Reports templates) and `8ca8b36` (CI passing).

This master document gives the whole picture in one read. Each section ends with links to the detailed documents, which carry the exact formulas, tables and tutorials:

Document	Contents
ARCHITECTURE.md	Layers, apps, design decisions, frontend, request trace, performance
DATABASE_AND_MIGRATIONS.md	SQLite configuration, data model, migrations tutorial on the real files
SECURITY.md	Authentication, sessions, authorization, hardening, audit, gaps
SIMULATION_AND_RISK_METHODOLOGY.md	Every formula, a worked numerical example, recommendations
TESTING_AND_QUALITY_ASSURANCE.md	What was run, what is recorded, what is not covered
DEVELOPMENT_AND_DEPLOYMENT.md	Set-up, configuration, Git/GitHub, CI, hosting, troubleshooting
LEARNING_GUIDE.md	A ten-stage study path with exercises
GLOSSARY.md	Terms and where the code defines them
diagrams/	Architecture, request lifecycle, data flow, authentication, simulation workflow, propagation, ERD, CI/CD
references/	Inventories: models/tables, routes, migrations, dependencies, environment variables, tests

Earlier project documents in `docs/` (requirements, architecture, data dictionary, threat model, test plan, user guide, deployment, engineering assessment, redesign acceptance) remain valid and are cited where relevant; this package does not replace them.

How to read the status labels

Every feature in this documentation carries one of five labels:

- **Implemented and verified** — code exists and a check passed on 2026-11-10 (the test suite in the working tree, or a command run for this documentation).
- **Implemented but unverified** — code exists, but no check was run for this documentation (results may be recorded earlier in the README).
- **Partial** — some of the stated capability exists.
- **Planned** — designed or recorded as future work in the project's documents, no code.
- **Not implemented** — absent, and not planned in the repository.

---

1. Purpose

What it is. CodeGuard is an educational web application that models how a compromise at one point of a software supply chain can spread to later (or earlier) points, and how security controls change the outcome. The README states the boundary plainly: it "never scans, attacks or connects to real systems, its scenarios are hypothetical, and its numbers are illustrative model outputs".

Why it exists. Supply-chain attacks are hard to reason about because the damage depends on paths between systems, not on any one system. CodeGuard turns that into an explicit, inspectable graph with deterministic rules, so a student or analyst can ask "what if" and see exactly why the answer changed.

How it works, in one paragraph. An analyst picks a threat scenario (an entry stage, a base likelihood, severity, impact and a set of directed propagation rules), chooses which security controls are enabled, and runs a simulation. The engine walks the graph twice, once without controls (*inherent*) and once with them (*residual*), scores every stage, aggregates an overall score, explains each decision in plain language, and recommends the controls that would lower the residual risk most. The run is stored with a complete snapshot of its inputs, rendered as a report with charts, and can be shared, exported, compared and turned into remediation work.

1.1 The five supply-chain stages

Seeded by `threats/seed_data.py` (`STAGES`) and stored in `threats_supplychainstage`:

Position	Key	Name	Weight	Typical assets (seed text, abbreviated)
1	`ide`	IDE / Developer Environment	1	developer accounts, local checkouts, SSH keys and tokens, IDE extensions
2	`vcs`	Version Control	2	repositories, branches, commits, pull requests, permissions
3	`cicd`	CI/CD Pipeline	2	build jobs, runners, pipeline credentials, release workflows
4	`registry`	Software Registry	2	published packages, container images, publishing credentials
5	`production`	Production	3	deployed services, runtime configuration, production credentials, customer data

Position orders the stages and classifies edges (forward, bypass, reverse). Weight expresses criticality in the overall score. Status: **implemented and verified** (`test_seed_creates_reference_data`).

1.2 Feature status overview

Feature	Status	Evidence / note
Five-stage model, control catalogue (14), six system scenarios	Implemented and verified	seed tests; seeded on a scratch DB
Scenario library: create, edit (versioned), archive/restore, delete, clone, filters	Implemented and verified	`threats/tests/test_scenarios.py`
Graph validation (self-loops, duplicates, unreachable rules) in form, engine and schema	Implemented and verified	`test_invalid_graph_rejected`, `TestConstraints`, `TestValidation`
Deterministic simulation engine (inherent + residual)	Implemented and verified	85 engine/risk tests; worked example recomputed by hand
Plain-language explanations per stage and per edge	Implemented and verified	`test_explanation_mentions_entry_and_scores`
Recommendations and leave-one-out control contributions	Implemented and verified	`test_recommendations.py`
Run persistence with reproducible snapshots	Implemented and verified	`test_snapshot_reproduces_the_stored_result`
Result pages (summary, stages, paths, controls, method)	Implemented and verified	`test_result_sections_and_report_document`
Matplotlib charts (overall, stages, outcomes, comparison, risk distribution)	Implemented and verified	`test_owner_sees_report_with_charts`, palette test

Feature	Status	Evidence / note
SVG pipeline and dashboard/analysis charts	Implemented and verified	`test_pipeline_geometry`, `test_svg_chart_geometry`
Run comparison (before/after)	Implemented and verified	`TestComparison`
CSV export (run and history) with formula-injection protection	Implemented and verified	`TestReportAccess`
Reports area (`/reports/` list and printable document)	Implemented and verified	templates were git-ignored until commit `eef6716`; a fresh clone of that commit passes all 337 tests
Report sharing (owner-only toggle, share-by-default preference)	Implemented and verified	`test_shared_report_visible_read_only_to_viewer`, `test_share_by_default_shares_new_reports`
Remediation plan with verification by later runs	Implemented and verified	`remediation/tests`
Overview dashboard, supply-chain map, stage pages, risk analysis	Implemented and verified	`core/tests/test_core.py`, `test_areas.py`
Notifications derived from existing records	Implemented and verified	`test_notifications_from_real_records`
Settings: profile, account, appearance, typography, display, accessibility, notifications, privacy, security, sessions, organization, users	Implemented and verified	`accounts/tests/test_settings.py`
Three roles with server-side checks; object-level access	Implemented and verified	`TestRoles`, IDOR tests
Registration policy (open/domain/closed) and deploy warning	Implemented and verified	`TestRegistrationPolicy`; `ci_deploy_check.py` output
Sign-in throttling	Implemented and verified	`TestLoginThrottling`
Session revocation ("sign out other sessions")	Implemented and verified	`test_signing_out_other_sessions`
Append-only audit trail, retention command	Implemented and verified	`audit/tests`
CSP and security headers	Implemented and verified	`test_security_headers_present`
Health check, request IDs, JSON logs	Implemented and verified	`core/tests/test_core.py`
Online SQLite backup	Implemented and verified (command); restore not automated	`test_backup_db_writes_a_verified_copy_and_rotates`
Personal data export (JSON)	Implemented and verified	`test_data_export_contains_only_own_data_and_is_audited`
Light/dark themes, accent, surface, text size, density, reduced motion, contrast	Implemented and verified (server side); browser rendering unverified here	`test_preferences_are_rendered_on_the_html_element`, design tests
WCAG 2.2 AA automated audit (axe-core, 47 pages)	Implemented but unverified	README records 0 violations (2026-10-28); not re-run
Live HTTP authorization matrix	Implemented but unverified	README records "all checks passed"; not re-run
GitHub Actions workflow	Implemented and verified	first passing run: run #7 on commit `8ca8b36`, 2026-11-10, 1 min 5 s; runs #1–#6 failed (missing Reports templates, then a Markdown format check)
Jenkins pipeline	Implemented but unverified	**never executed on Jenkins**; commands rehearsed only

Feature	Status	Evidence / note
Production deployment guide	Partial	settings and commands tested; proxy/systemd examples untested; no server exists
Hosting on PythonAnywhere	Not implemented	discussed as a possible option only
Multi-tenant organizations	Planned (decision pending)	`docs/engineering-assessment.md` H2
Asset-level graph	Planned	assessment M7
Uncertainty / Monte-Carlo mode	Planned	assessment M8
MFA for CodeGuard accounts	Not implemented	—
PostgreSQL support	Not implemented	settings are generic; untested
Background job queue, REST API, PDF generation	Not implemented	the report document is printed by the browser (`data-print` button)
Screen-reader testing	Not implemented	README states it was never done

---

2. Technology stack

What it is and why. A deliberately small, conventional stack: everything needed ships with Python plus three runtime packages.

Layer	Technology	Version	Role in CodeGuard
Language	Python	≥ 3.12 (verified 3.12.10)	everything server-side
Framework	Django	5.2.18	ORM, migrations, templates, auth, sessions, CSRF, admin, management commands
Database	SQLite	bundled with Python	one file, WAL mode
Charts	Matplotlib (Agg backend)	3.11.2	PNG charts rendered in memory
Configuration	python-dotenv	1.2.4	loads `.env`
CSS/JS	Bootstrap + Bootstrap Icons (vendored), one hand-written `app.js`, one `codeguard.css`	not recorded	layout, components, progressive enhancement
Fonts	Google Fonts (Instrument Sans, JetBrains Mono, Newsreader)	—	the only third-party request
Tests	pytest, pytest-django, pytest-cov, coverage	9.1.1 / 4.14.0 / 7.1.0 / 7.17.0	337 tests
Quality	Ruff	0.17.2	lint (incl. security rules) and format
Supply-chain hygiene	pip-audit	2.10.1	dependency vulnerability scan
CI	Jenkins (Declarative `Jenkinsfile`), GitHub Actions	—	quality gates; no deployment
Optional verification	Node ≥ 22 + axe-core 4.10.2 + Chrome; Python `requests`	—	accessibility and live authorization checks

Not used: any JavaScript framework or bundler, Celery/Redis, Docker, a REST framework, a WSGI server package (installed separately at deployment). Details: references/dependencies.md.

3. Directory structure

```
CODEGUARD/
├── codeguard/            project package: settings.py, test_settings.py, urls.py, wsgi.py, asgi.py
├── accounts/             roles, registration, throttling, session epoch, settings pages, preferences
├── audit/                AuditEvent, record_event, audit log, purge_audit_events command
├── threats/              stages, controls, scenarios, rules, forms, seed data, seed command
├── simulation/           domain, engine, propagation, risk, recommendations, serialization,
│                         services, views, charts (Matplotlib), visual (SVG), exports (CSV)
├── remediation/          RemediationItem, plan and detail views, verification
├── core/                 landing, dashboard, analytics, notifications, help, healthz, errors,
│                         CSP + request-ID middleware, theme script, SVG charts, template tags,
│                         backup_db command
├── templates/            base.html, layouts/, components/, partials/, illustrations/, and one
│                         folder per area (core, dashboard, supply_chain, risk, threats,
│                         simulation, reports*, remediation, audit, settings, registration, errors)
├── static/               css/codeguard.css, js/app.js, images/favicon.svg, vendor/ (Bootstrap)
├── tests/                cross-app integration tests and CI-script tests
├── scripts/              ci_local.ps1/.sh, ci_deploy_check.py, ci_dependency_audit.py,
│                         rehearse_jenkinsfile.py
├── tools/                a11y/ (Node browser checks), authz_live.py
├── docs/                 project documents; docs/complete/ is this package
├── .github/              workflows/ci.yml, issue and PR templates
├── Jenkinsfile, manage.py, conftest.py, pytest.ini, pyproject.toml
├── requirements.txt, requirements-dev.txt, .env.example
└── README.md, CONTRIBUTING.md, DESIGN.md, PLAN.md
```

`*` `templates/reports/` exists on disk but is excluded by `.gitignore` (§16). Each app follows the Django layout (`models.py`, `views.py`, `urls.py`, `forms.py`, `admin.py`, `migrations/`, `tests/`) plus a `services.py` for business rules. Local-only items not described here: `db.sqlite3`, `.env`, `.venv/`, `backups/`, `reports/`, caches, and a stray clone folder `codeguard-clean/`.

---

4. Backend

What it is. A Django monolith with six project apps (see ARCHITECTURE.md §2).

Why this shape. One deployable unit is the simplest thing that works for a single team, and Django supplies authentication, forms, an ORM, migrations and an admin without extra services.

How it works. Requests pass a ten-entry middleware chain (`codeguard/settings.py` `MIDDLEWARE`), reach function-based views, which call service functions, which use the ORM. The simulation engine is a separate pure-Python layer that services call with immutable dataclasses. The dependency rule, quoted from `simulation/domain.py`: "These classes deliberately have no Django dependency… a simulation cannot accidentally read or write the database."

How CodeGuard uses it — the main execution path, end to end.

1. `POST /simulations/new/` → `simulation.views.configure` (decorated `analyst_required`).
2. `SimulationForm` validates ranges (likelihood 0–100, severity/impact 1–5, threshold 1–100) and limits the scenario to *active scenarios visible to this user*.
3. `services.run_simulation` builds a `SimulationInput` (`build_input`) from all stages, the scenario's rules and the chosen controls, and calls `engine.validate_input` — an invalid configuration raises `SimulationConfigError` and **no run is created**.
4. A `SimulationRun` row is created with status `pending` and a version-2 `input_snapshot`.

5. Inside `transaction.atomic()`, `_execute` runs `engine.simulate`, `recommend` and `control_contributions`, bulk-creates `SimulationStageResult` and `PropagationEvent` rows, stores `result` JSON and headline scores, and sets status `completed`.
6. Any exception rolls back the children and marks the run `failed` with a generic message; an audit event is written either way (`simulation_run` or `simulation_failed`).
7. If the user prefers it, the report is shared at once. The browser is redirected to `/simulations/<uuid>/`.
8. The result page and its chart PNGs read **only stored data**.

Sequence diagram: diagrams/simulation-workflow.md.

Source files. `simulation/views.py`, `forms.py`, `services.py`, `engine.py`, `recommendations.py`, `serialization.py`, `models.py`.

Example. `threats/services.save_scenario` is the same pattern on the scenario side: one atomic function that saves the scenario (incrementing `version` on edits, forcing `private` for non-admins), its rule formset and its control links.

Limitations. Synchronous execution in the request (fine at millisecond cost); function views with inline context building; no API layer.

---

5. Frontend

What it is. Server-rendered Django templates styled by one design-token stylesheet and enhanced by one small script.

Why. No build step, works without JavaScript for nearly everything, and keeps all logic on the server where it is tested.

How it works. `templates/base.html` loads Google Fonts, vendored Bootstrap CSS/Icons, `static/css/codeguard.css`, the Bootstrap bundle and `static/js/app.js`, and writes the pre-paint theme script inline (allowed by its CSP hash). User preferences become `data-*` attributes on `<html>`, which the CSS token blocks read. `app.js` adds confirmation dialogs, loading states, print buttons, accessible scroll regions, the theme toggle, chart tooltips, the configuration stepper and chart re-colouring for the active theme.

How CodeGuard uses it. `DESIGN.md` is the design authority; `CLAUDE.md` adds "Never hardcode colors; use the CSS variables", and `core/tests/test_design.py` enforces it automatically. The five-stage pipeline diagram is an SVG whose geometry is computed in `simulation/visual.build_pipeline` and drawn by `templates/components/pipeline.html`.

Source files. `templates/`, `static/css/codeguard.css`, `static/js/app.js`, `core/theme.py`, `core/templatetags/codeguard_tags.py`, `accounts/preferences.py`.

Limitations. The vendored Bootstrap version is not recorded; Google Fonts is a third-party request on every page; browser behaviour of `app.js` is tested only by the optional Node tools. More: ARCHITECTURE.md §4.

---

6. Database

What it is. One SQLite file accessed through the Django ORM: 12 CodeGuard models in 13 tables, plus 10 Django framework tables, created by 15 project migrations.

Why. Zero administration for a single-team academic tool.

How it works. WAL journal, IMMEDIATE transactions and a 20-second busy timeout let several worker processes share the file. Integrity rules (ranges, uniqueness, no self-loops, private scenarios must have an owner, one active remediation item per control and scenario) are database constraints as well as form rules. History is denormalised so reports outlive scenarios.

How CodeGuard uses it. Reference data (stages, controls, system scenarios) is loaded by the idempotent `seed_codeguard` command rather than by migrations. JSON columns hold snapshots, results and audit metadata.

Example. The data migration `accounts/migrations/0006_retire_neutral_surface.py` moves users off a retired preference value before narrowing the choices; it was reversed and re-applied on a scratch database for this documentation.

Limitations. One writer at a time, same-host only, no tested PostgreSQL path, and audit-table immutability is enforced in Python only.

More: DATABASE_AND_MIGRATIONS.md, diagrams/erd.md, references/models-and-tables.md, references/migrations.md.

---

7. Authentication, authorization and security

What it is. Django session authentication with three CodeGuard roles (admin, analyst, viewer) stored in `Profile.role`, server-side role decorators, object-level scoped querysets, and a hardened HTTP layer.

Why. Scenarios and reports can be private; administrators manage shared content and users; the audit trail must be trustworthy.

How it works.

- *Sign-in* uses `ThrottledAuthenticationForm`, which refuses before checking the password when a username has 5 failures, or an IP has 20, within 15 minutes, counting audit events.
- *Registration* never lets the user choose a role; `CODEGUARD_REGISTRATION` decides who may register at all.
- *Authorization* is checked in every protected view (`login_required`, `analyst_required`, `admin_required`, and `can_edit`/`can_delete`); objects are fetched through `visible_scenarios`, `accessible_runs` and `items_for`, and a miss is a 404.
- *Sessions* last 8 hours and end at browser close; "sign out other sessions" increments a per-user epoch checked by `SessionEpochMiddleware`.
- *Headers*: strict CSP with a script hash, `X-Frame-Options: DENY`, `nosniff`, same-origin referrer and opener policies, a restrictive Permissions-Policy, HTTPS/HSTS/secure cookies by default when DEBUG is off.

How CodeGuard uses it. See the role matrix in SECURITY.md §3.1. Viewers can read shared content and their own audit events; analysts create private scenarios, run simulations and manage their remediation plan; admins also manage shared scenarios, users and the organization page and see every audit event. Admins do **not** get access to other users' private scenarios or runs (tested).

Example. `GET /simulations/<another user's private run>/` → `get_accessible_run` finds nothing → 404, the same response as for a non-existent ID (`test_other_user_cannot_open_private_report_chart_or_csv`).

Honest gaps. No MFA for CodeGuard accounts; per-IP throttling becomes one shared bucket behind a reverse proxy because only `REMOTE_ADDR` is trusted; open registration by default with instance-wide sharing (single-tenant by design); `/admin/` is outside the CSP and the audit trail; no penetration test. Full list: SECURITY.md §8. Diagram: diagrams/authentication.md.

---

8. Threat modelling in CodeGuard

Two different things carry this name; keep them apart.

8.1 Threat scenarios (the product's subject matter). A `ThreatScenario` is a hypothetical attack: an entry stage, base likelihood (0–100), severity and impact (1–5), a propagation threshold (default 50), narrative fields (business impact, assets, assumptions, preventive and detective measures), and a set of `PropagationRule` edges with probabilities and rationales. Scenarios are shared or private, active or archived, versioned on every edit, and the six seeded system scenarios cannot be deleted. The seeded set covers every stage as an entry point:

Scenario	Entry	L / S / I	Notable edges
Compromised Developer Account	IDE	70 / 4 / 4	forward chain + IDE → CI/CD bypass
Compromised Version Control Account	VCS	60 / 4 / 4	VCS → IDE reverse
Compromised CI/CD Pipeline	CI/CD	55 / 5 / 5	CI/CD → VCS reverse, CI/CD → Production bypass
Malicious Dependency	Registry	50 / 4 / 4	reverse edges to IDE and CI/CD, a CI/CD ↔ Registry cycle
Compromised Software Registry	Registry	45 / 5 / 5	Registry → CI/CD reverse, CI/CD → Production bypass
Production Credential Exposure	Production	65 / 5 / 5	only reverse edges (30 %, 35 %)

Status: **implemented and verified** (`test_every_seeded_scenario_simulates_successfully`).

8.2 The threat model of the application itself. `docs/threat-model.md` applies STRIDE to CodeGuard (assets, actors, trust boundaries, mitigations, residual risks). This package's SECURITY.md maps those mitigations to code and tests.

---

9. Simulation and risk scoring

What it is. A deterministic graph model (`MODEL_VERSION = "1.0"`).

How it works — the formulas (complete definitions in SIMULATION_AND_RISK_METHODOLOGY.md §2):

```
coverage(s)          = 100 × (1 − ∏(1 − εᵢ/100))       per stage, per mode (prevent / mitigate)
effective(u→v)       = p × (1 − P(v)/100)
propagates           ⇔ effective ≥ τ                    (else blocked if p ≥ τ, else below threshold)
L_entry              = L₀ × (1 − P(entry)/100)
λ(v)                 = best successful path product, or strongest failed attempt
raw(λ)               = λ × (severity/5) × (impact/5)
residual_risk(s)     = raw(λ_residual(s)) × (1 − M(s)/100)
overall              = Σ risk(s)·w(s) / Σ w(s)
bands                low < 15 ≤ moderate < 30 ≤ high < 50 ≤ critical
```

The traversal is breadth-first, evaluates every edge at most once (so cycles terminate), and picks the most likely path with a max-product Dijkstra variant. The inherent pass ignores all controls; the residual pass applies preventive coverage to edges and to the entry stage, and mitigating coverage to stage risk.

Worked example (summary). "Compromised Developer Account" with MFA, protected branches and SCA: inherent overall **26.6 (moderate)**, residual overall **3.3 (low)**; only the entry stage stays compromised; IDE → Version Control is blocked (85 % → 17 %), IDE → CI/CD is below the threshold (40 %). Step-by-step numbers: SIMULATION_AND_RISK_METHODOLOGY.md §3.

Limitations (also shown on every report's "Inputs and method" tab): fixed threshold, not a calibrated prediction; illustrative effectiveness values; independent controls; single best path, not a sum; no telemetry. Further observed limitations (discontinuity at the threshold, scenario- wide severity) are listed in §7 of the methodology document.

---

10. Controls and recommendations

What it is. A catalogue of 14 controls (`threats/seed_data.CONTROLS`): 11 preventive, 2 detective (`sca`, `runtime-monitoring`) and 1 recovery (`incident-response`). Each has an effectiveness parameter, the stages it protects, effort, priority, what it addresses, and since migration `threats.0003` an `effectiveness_basis` and `limitations` text, so the number is never presented without its assumption (`test_every_seeded_control_states_its_basis_and_limitations`).

Why. Students need to see that controls differ in what they do (prevent vs mitigate), where they act, and how much they cost, and that a stated number is an assumption.

How it works. Preventive controls reduce the probability of entering the stages they protect; detective and recovery controls reduce the residual risk of a reached stage. Recommendations are computed by **re-running the engine** with each disabled, relevant candidate added, ranking by the exact modelled reduction (ties: scenario-applicable first, lower effort, higher priority, name), and keeping five. Contributions of enabled controls are computed leave-one-out.

How CodeGuard uses it. The "Controls and recommendations" tab shows both lists with reasons; "Add to plan" turns a recommendation into a `RemediationItem` (§11). The dashboard aggregates recommendations across a user's last ten runs (`core.services.top_recommendations`).

Example. In the worked example the top recommendation is dependency pinning (−1.9 points), because it also protects the entry stage.

Limitations. Reductions are model outputs, not expected real-world effects; contributions do not add up because controls interact.

---

11. Remediation

What it is. A per-user plan of controls the user committed to adopt (`remediation` app).

How it works. `remediation.services.track` accepts only controls that appear in the run's stored recommendations, stores the baseline risk and expected reduction, and refuses duplicates (service check plus a partial unique index). Items move between open, in progress, done and risk accepted; `closed_at` is set and cleared accordingly; every change is audited. *Verification* is derived, not stored: the owner's latest later completed run of the same scenario with the control enabled (`services.verification`). The plan can be filtered by status and sorted by due date, modelled impact or recency; overdue and unscheduled counts are shown.

Status. Implemented and verified (`remediation/tests/test_remediation.py`, 11 tests). Viewers cannot create or edit items.

---

12. Reporting, charts and analytics

Output	Where	Technology	Status
Result tabs: summary (with affected assets), stage analysis, propagation paths, controls and recommendations, inputs and method	`/simulations/<id>/<section>/`	templates + stored rows	verified
Charts: overall inherent vs residual, risk by stage, outcomes by type	`/simulations/<id>/chart/<kind>.png`	Matplotlib, in memory	verified
Before/after comparison and its chart	`/simulations/compare/?a=&b=`	Matplotlib	verified
Reports archive and printable assessment document	`/reports/`, `/reports/<id>/`	templates	verified (tracked since commit `eef6716`)
CSV export of a run and of one's history	`export.csv` routes	`csv` + `safe_cell`	verified
Overview: run totals, averages, band distribution, stage strip, remediation summary, recommended controls	`/overview/` (+ PNG)	ORM aggregates, SVG bars, Matplotlib	verified
Supply-chain map and stage pages	`/supply-chain/`	`core/analytics.py`	verified
Risk analysis: trend of the 24 latest runs, exposure by stage, control effects	`/risk/`	`core/svgcharts.py`	verified

Charts take their colours from the page's CSS tokens via `?palette=`, falling back to the light palette, and every chart has a text or table alternative. Charts are only served for runs the user may access, with `Cache-Control: private`.

---

13. Audit trail

What it is. `audit.AuditEvent`, an append-only record of 25 kinds of actions.

Why. Accountability for security-relevant actions, evidence for throttling, and a source for the security page and notifications.

How it works. `audit.services.record_event(actor, action, target, request, **metadata)` stores the actor and username, the object type/id/repr, sanitised metadata (keys containing password, secret, token, session, csrf, key or credential are dropped; values truncated) and `REMOTE_ADDR`. Updates and deletes raise at the model and queryset level; deleting a user keeps their events. `purge_audit_events --days N` (minimum 30) is the only way to remove history, and it records itself.

How CodeGuard uses it. `/audit/` (timeline or table, filters) shows administrators every event and others their own. `accounts/throttling.py` counts `login_failed` events; `/settings/security/` lists recent account events; `core/notifications.py` derives "shared report" and "security" notifications from events.

Limitations. No database triggers, so raw SQL could still alter rows; Django admin actions are not recorded here; IP addresses are the proxy's behind a proxy.

---

14. Testing and quality assurance

Status on 2026-11-10. 337 tests exist and **all 337 passed** in the working tree; coverage 96 % overall and 99 % on the four core engine modules (gate: 85 %); Ruff clean; no missing migrations; `check --deploy` has no errors. A fresh clone of `8f5d092` failed 9 tests because the Reports templates were git-ignored; after the fix in commit `eef6716` a fresh clone passes all 337. Browser accessibility checks, the live authorization matrix and the Jenkins rehearsal were not re-run (their earlier results are recorded in the README). `pip-audit` was re-run: no known vulnerabilities. GitHub Actions passed on its run #7 (commit `8ca8b36`).

Strategy: pure engine unit tests, service/view tests through Django's test client, integration tests, tests that enforce the design system on CSS and templates, Python-level accessibility tests and optional browser tools. Details and gaps: TESTING_AND_QUALITY_ASSURANCE.md, references/tests.md.

---

15. Git, GitHub, Jenkins and CI/CD

- **Git/GitHub.** 10 commits on `main` (2026-10-27 → 2026-11-10), one author (`bsclmr134322-collab`), remote `github.com/bsclmr111626/software-engineering-project`. Contribution guide, issue and PR templates exist.
- **Jenkins.** `Jenkinsfile` defines seven stages: checkout, environment setup, static checks, database preparation (migration check, migrate, seed twice), tests with JUnit and coverage, core coverage gate, security checks (deploy check and pip-audit with SUCCESS/UNSTABLE/FAILURE mapping), then archiving and cleanup. **It is configured but has never been executed on a Jenkins server.** `scripts/rehearse_jenkinsfile.py` extracts and runs its commands on a fresh clone ("commands only; not a Jenkins execution"); the README records SUCCESS on 2026-10-26. Before commit `eef6716` the rehearsal would have failed in the test stage (§16, gap 1).
- **GitHub Actions.** `.github/workflows/ci.yml` mirrors the gates on every push to `main` and every pull request. Runs #1–#6 failed; run #7 on commit `8ca8b36` (2026-11-10) **passed** every gate in 1 min 5 s. The two causes of the earlier failures were the git-ignored Reports templates (fixed in `eef6716`) and Ruff's format check rewriting Python code quoted in the new Markdown docs (fixed in `8ca8b36` by excluding `*.md` from `ruff format`).
- **CD.** None: no pipeline builds a release or deploys.

Diagram: diagrams/ci-cd.md; details: DEVELOPMENT_AND_DEPLOYMENT.md §3–4.

---

16. Configuration and environment variables

All settings that vary between environments are environment variables, optionally loaded from `.env` (copied from `.env.example`). The key ones: `DJANGO_DEBUG`, `DJANGO_SECRET_KEY` (required when DEBUG is off), `DJANGO_ALLOWED_HOSTS`, `DJANGO_CSRF_TRUSTED_ORIGINS`, `DJANGO_DB_PATH`, `CODEGUARD_REGISTRATION` (+ `_DOMAINS`), `CODEGUARD_DEFAULT_ROLE`, the three `CODEGUARD_LOGIN_*` throttling values, the email variables, the transport-security switches and the two logging variables. Invalid enumerations stop start-up with `ImproperlyConfigured`. `DJANGO_HSTS_INCLUDE_SUBDOMAINS` is read by settings but missing from `.env.example`. Complete table: references/environment-variables.md.

Known gaps found while documenting

1. **Reports templates were not version-controlled — fixed in commit `eef6716`.** The `.gitignore` rule `reports/`, meant for CI artefacts, also matched `templates/reports/`, so `list.html` and `document.html` were never committed: a clone of `8f5d092` returned 500 on `/reports/` and failed 9 tests. The rule is now

`/reports/` and both templates are committed; a fresh clone of `eef6716` passes all 337 tests. Not yet pushed at the time of writing.

2. **Per-IP sign-in throttling behind a proxy** counts all users as one IP.
3. **`requests` is unpinned** although `tools/authz_live.py` imports it.
4. **`DJANGO_HSTS_INCLUDE_SUBDOMAINS` is undocumented** in `.env.example`.
5. **A route-ordering edge case:** a scenario whose slug is `controls` could never be opened, because `threats/controls/` is matched first and `unique_slug` does not reserve it.
6. **Comment drift:** `core/middleware.py` says the Google Fonts allowance is for "Source Serif 4", but `templates/base.html` loads Instrument Sans, JetBrains Mono and Newsreader.

---

17. Installation and deployment

- **Local installation**: the README Quick start (clone, venv, `pip install -r requirements.txt`, copy `.env.example`, `migrate`, `seed_codeguard --demo-users`, `runserver`). On this machine, call `.venv\Scripts\python.exe` because Python is not on `PATH`.
- **Production**: `docs/deployment.md` describes one Linux host with a WSGI server (gunicorn or waitress, installed separately), a TLS reverse proxy serving `collectstatic` output, SQLite on local disk, health checks, backups, retention and upgrades. The application-side parts are tested; the systemd and nginx snippets are examples that were never run. No production or staging server exists.
- **Hosting options**: a self-managed Linux VM (the documented path); PythonAnywhere's free tier was discussed as a possible option and **has not been set up or tried**; there is no Dockerfile.

Details and troubleshooting: DEVELOPMENT_AND_DEPLOYMENT.md.

---

18. Performance

Observed from the code; nothing was benchmarked for this documentation.

- A simulation costs 1 + (number of recommendation candidates) + (number of enabled controls) engine runs over a 5-node graph — milliseconds — so synchronous execution is acceptable (assessment L1).
- Pages use `select_related`/`prefetch_related`; user preferences are cached per request.
- Dashboard and analytics recompute SQL aggregates over all accessible runs on every request; there is no cache.
- The remediation plan has an N+1 pattern (`verification()` per item, up to 50 runs each).
- SQLite allows one writer at a time; the README's 200-run concurrency experiment (2026-10-26) is the only load evidence.

---

19. Maintainability

Strengths: a pure, well-tested engine; one formula per function with docstrings; enumerations for every state; constraints in both forms and schema; versioned snapshots; consistent service-layer access checks; Ruff with security and Django rules; design rules enforced by tests; CI scripts written in Python so they can be unit-tested. Weak spots: long settings views, inline context building in views, vendored assets without versions, and version-control hygiene (the Reports templates were ignored templates). The engine's generic `Stage`/`Edge` model is the intended extension point for an asset graph (assessment M7).

---

20. Troubleshooting (most common)

Symptom	Fix
`DJANGO_SECRET_KEY must be set…`	create `.env` from `.env.example`
`no such table`	`python manage.py migrate`
No scenarios	`python manage.py seed_codeguard`
500 / `TemplateDoesNotExist: reports/list.html`	a checkout older than `eef6716` lacks the Reports templates (§16)
"Too many failed sign-in attempts"	wait 15 minutes or adjust `CODEGUARD_LOGIN_*`
No styles in production	`collectstatic` and serve `/static/` from the proxy
`python` not found on this PC	use `.venv\Scripts\python.exe`

Full table: DEVELOPMENT_AND_DEPLOYMENT.md §7.

---

21. Limitations and future work

Model. Deterministic single-point estimates; illustrative parameters; independent controls; best path only; a hard threshold; scenario-wide severity and impact; a fixed five-stage model. Future work recorded by the project: an uncertainty (Monte-Carlo) mode (M8) and an optional asset layer with trust relationships (M7).

Product and platform. Single-tenant sharing (organizations need a product decision, H2); no MFA for CodeGuard accounts; no API; no background jobs; SQLite only.

Verification. Jenkins never executed (GitHub Actions does pass, since run #7); no real deployment; no screen-reader test; no penetration test; browser and authorization checks last run 2026-10-26/28.

Immediate fixes suggested by this documentation pass: the Reports templates and ignore rule were fixed in commit `eef6716`; still open: add a trusted-proxy option for `client_ip`; pin `requests` in a tools requirements file; add `DJANGO_HSTS_INCLUDE_SUBDOMAINS` to `.env.example`; reserve the `controls` slug; correct the font comment in `core/middleware.py`.

---

22. Glossary and learning guide

Terms: GLOSSARY.md. A ten-stage study path from "run it" to "explain it", with exercises against the real code: LEARNING_GUIDE.md.

Appendix — How this documentation was verified

- Every file path, class and function named here was checked against the repository (see the final verification notes in the hand-over report).
- Formulas were copied from `simulation/risk.py`, `propagation.py` and `engine.py`; the worked example was computed by the real engine on a seeded scratch database and recomputed by hand.
- Test counts come from `pytest --collect-only`; results from runs on 2026-11-10.
- Table names come from a database created from scratch with `migrate`.
- The local `.env` file was not read. No application code was changed.

Part II — Detailed documents

Architecture

CodeGuard is a **server-rendered Django monolith**: one deployable unit, six project apps, one SQLite database, no JavaScript build step and no external services at runtime apart from Google Fonts. Diagrams: architecture, request lifecycle, data flow.

1. Layers

Layer	What lives there	Rule
Presentation	`templates/` (Django templates), `static/css/codeguard.css`, `static/js/app.js`, vendored Bootstrap	Templates only display; geometry for diagrams is computed in Python (`simulation/visual.py`, `core/svgcharts.py`)
Views	`*/views.py`, `accounts/settings_views.py`	Function-based views. Authentication and role decorators on every protected view; no business rules beyond input parsing
Services	`simulation/services.py`, `threats/services.py`, `remediation/services.py`, `core/services.py`, `core/analytics.py`, `core/notifications.py`, `audit/services.py`	Visibility rules, transactions, persistence, aggregation
Domain engine	`simulation/domain.py`, `engine.py`, `propagation.py`, `risk.py`, `recommendations.py`, `serialization.py`	Pure Python; imports no Django module; never touches the database
Persistence	`*/models.py`, `*/migrations/`	Django ORM, constraints in the schema

The direction of dependencies is strictly downward. The one upward-looking import is `threats/models.py` importing `simulation.propagation.edge_kind`, a pure function used for the `PropagationRule.kind` property.

2. The apps

App	Responsibility	Main files
`accounts`	Roles (`Profile`), registration policy, sign-in throttling, session revocation, every Settings page, user preferences, deploy check `codeguard.W001`	`models.py`, `permissions.py`, `throttling.py`, `middleware.py`, `signals.py`, `preferences.py`, `settings_views.py`, `checks.py`
`audit`	Append-only `AuditEvent`, `record_event`, the audit log page, retention command	`models.py`, `services.py`, `views.py`, `management/commands/purge_audit_events.py`
`threats`	Stage catalogue, control catalogue, scenarios and propagation rules; seed data	`models.py`, `forms.py` (graph validation), `services.py`, `seed_data.py`, `management/commands/seed_codeguard.py`
`simulation`	Engine, run persistence, result/report views, Matplotlib charts, CSV exports, comparisons	see the Domain row above, plus `services.py`, `views.py`, `charts.py`, `exports.py`, `visual.py`, `report_urls.py`
`remediation`	Remediation plan created from recommendations; verification by later runs	`models.py`, `services.py`, `views.py`
`core`	Landing page, overview dashboard, supply-chain map, risk analysis, help/methodology, notifications, health check, error pages, CSP and request-ID middleware, theme script, template tags, backup command	`views.py`, `services.py`, `analytics.py`, `notifications.py`, `middleware.py`, `observability.py`, `theme.py`, `svgcharts.py`, `templatetags/codeguard_tags.py`

`core` has no models. The Reports area (`/reports/`) is not an app: its URLs live in `simulation/report_urls.py` and its views in `simulation/views.py` because they read simulation models.

3. Key design decisions (with the reason found in the code)

1. **Engine separated from Django.** `simulation/domain.py`: "These classes deliberately have no Django dependency… guarantees that a simulation cannot accidentally read or write the database." It makes the 85 engine and risk tests fast and database-free (`simulation/tests/factories.py`).
2. **Stored snapshots, not live joins, for history.** `SimulationRun` stores `input_snapshot` and `result` JSON plus denormalised scenario name/slug/version, and stage results/events store stage names as text. Docstring: "editing or deleting the scenario later never changes a historical report."
3. **Charts rendered on the server.** `simulation/charts.py` uses Matplotlib's object-oriented API (`Figure` + `FigureCanvasAgg`) rather than `pyplot`, because it "keeps no global state, is safe in a multi-threaded server". PNGs are produced in memory and returned by access-controlled views, so no chart file or path ever exists.
4. **Synchronous runs.** A simulation runs inside the POST request. The engine runs once for the result plus once per recommendation candidate and once per enabled control, all in milliseconds; `docs/engineering-assessment.md` L1 records this as accepted.
5. **SQLite tuned for several workers.** `DATABASES["default"]["OPTIONS"]`: WAL journal, `transaction_mode: IMMEDIATE`, 20 s busy timeout, `synchronous=NORMAL`, foreign keys on.
6. **Single-tenant deployment.** "Shared" means every account on the instance; registration policy (`CODEGUARD_REGISTRATION`) is the tenancy control (assessment H2).
7. **Progressive enhancement.** `static/js/app.js` header: "Every feature also works without JS except the 'add rule' helper…, the account menu… and chart tooltips."

4. Frontend

- **Templates.** `templates/base.html` loads the CSS, the vendored Bootstrap bundle and `static/js/app.js`, and renders the pre-paint theme script via the `theme_init_script` tag. Layouts: `layouts/public.html`, `auth.html`, `app.html`, `settings.html`. Reusable pieces in `components/` (pipeline SVG, legend, pagination, messages, form field, status), `partials/` and inline SVG illustrations in `illustrations/`.
- **Design system.** `DESIGN.md` defines the tokens; colours exist only between `@tokens:start` and `@tokens:end` in `static/css/codeguard.css`. `core/tests/test_design.py` enforces the rules (no colour literals in templates or JS, no theme branching, both themes define the same tokens).
- **Preferences reach the CSS** as `data-*` attributes on `<html>` (`accounts.preferences.html_attributes`: `data-theme-pref`, `data-accent`, `data-surface`, `data-text-size`, `data-heading-font`, `data-density`, and optional `data-motion`, `data-contrast`, `data-links`).
- **Charts follow the theme.** `app.js renderCharts()` reads nine CSS variables (`PALETTE_TOKENS` order: surface, text, text-muted, border, border-strong, sev-low, sev-med, sev-high, sev-crit) and rewrites each chart `<img>` URL with `?palette=`; `charts.parse_palette` falls back to the light palette for anything malformed.
- **Two chart technologies.** Matplotlib PNGs on run pages, comparison and the dashboard distribution (`/overview/risk-distribution.png`); SVG geometry computed in Python (`core/svgcharts.hbars`, `line_chart`, `paired_columns`) on the overview and risk-analysis pages and the five-stage pipeline (`simulation/visual.build_pipeline`, drawn by `components/pipeline.html`). Charts have text/table alternatives (README "Accessibility").

5. End-to-end trace: opening a stored result and its chart

This complements the form-to-engine trace in simulation-workflow.

1. `GET /simulations/<uuid>/stages/` resolves to `simulation.views.detail(request, run_id, section="stages")`; an unknown section raises 404.
2. `services.get_accessible_run(user, run_id)` filters `Q(user=user) | Q(is_shared=True, status=COMPLETED)`; a miss (or a malformed UUID) is 404, so the existence of other users' private runs is not disclosed.
3. `_result_context` loads `stage_results` and `events` (ordered by position / order), counts outcomes for the chart's text alternative, finds the owner's previous completed run of the same scenario, builds the pipeline geometry from stored rows, adds unevaluated rules from the snapshot as "Not evaluated", reads `limitations` from the stored result, flags `scenario_changed` when the scenario's version moved on, and looks up the user's tracked remediation items.
4. The template emits `<img src="…/chart/stages.png" data-chart-palette=…>`; `app.js` swaps in the current theme palette.
5. `simulation.views.chart` re-checks access, refuses incomplete runs, calls `charts.stage_risk_chart(list(run.stage_results.all()), palette)` and returns `image/png` with `Cache-Control: private, max-age=300`.

6. Cross-cutting concerns

Concern	Mechanism	File
Request correlation	`RequestIdMiddleware` + `RequestIdFilter`; header `X-Request-ID`	`core/observability.py`
Logging	stdout/stderr, text or JSON (`CODEGUARD_LOG_FORMAT`)	`codeguard/settings.py` `LOGGING`
Audit	`record_event()` called from views, services and auth signals	`audit/services.py`
Shell context	`core.context_processors.shell` (preferences, nav section, unread count) and `accounts.context_processors.role`	
Errors	`handler400/403/404/500` and `CSRF_FAILURE_VIEW` render friendly templates without details	`core/views.py`
Health	`/healthz/` runs `SELECT 1`	`core/views.healthz`

7. Performance characteristics (observed from the code, not benchmarked here)

- Queries are bounded with `select_related`/`prefetch_related` on list pages; preferences are cached on the user object per request (`preferences_for`).
- `remediation.views.plan` calls `services.verification(item)` for every item on the page, each scanning up to 50 runs: an N+1 pattern that is fine for a page of 10–50 items.
- Dashboard and analytics aggregate in SQL over all accessible completed runs on every request; there is no caching layer.
- README records a concurrency test (4 processes × 50 runs on one SQLite file, 200/200 completed, 2026-10-26). It was not repeated for this documentation.

8. Maintainability

Strengths: small modules with docstrings that explain *why*; one formula per function; enums (`TextChoices`) for every state; constraints duplicated in forms and schema; a design-rule test suite; Ruff with security (`S`) and Django (`DJ`) rules. Weak spots: settings views are long (`accounts/settings_views.py`), views build context dictionaries inline, and the Reports templates were outside version control until commit `eef6716` (see the master document, "Known gaps").

Database and migrations

CodeGuard stores everything in one SQLite file through the Django ORM. This document explains the database configuration and the data model, then teaches Django migrations using CodeGuard's own 15 migration files. Inventories: models and tables, migrations; diagram: ERD.

Everything marked "verified" below was run on 2026-11-10 against a scratch database (`DJANGO_DB_PATH` pointed at a temporary file), never against the developer's `db.sqlite3`.

Part 1 — The database

1.1 SQLite and how CodeGuard configures it

What it is. SQLite is an embedded SQL database: a library inside the Python process that reads and writes one file. There is no database server.

Why CodeGuard uses it. The project is an academic, single-team tool. SQLite ships with Python, needs no installation and makes the whole application one directory plus one file. `codeguard/settings.py` points to `docs/architecture.md` ("Database: SQLite now, PostgreSQL when") for the recorded decision.

How it is configured (`codeguard/settings.py`):

```
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": os.environ.get("DJANGO_DB_PATH", str(BASE_DIR / "db.sqlite3")),
        "OPTIONS": {
            "transaction_mode": "IMMEDIATE",
            "timeout": 20,
            "init_command": "PRAGMA journal_mode=WAL; PRAGMA synchronous=NORMAL; PRAGMA foreign_keys=ON",
        },
    }
}
```

Option	Effect
`journal_mode=WAL`	Write-ahead log: readers keep reading while one writer commits. Creates `-wal` and `-shm` side files. Verified: `PRAGMA journal_mode` returns `wal`.
`transaction_mode: IMMEDIATE`	A transaction takes the write lock when it begins, so two writers queue instead of failing halfway with "database is locked".
`timeout: 20`	A waiting writer waits up to 20 s.
`synchronous=NORMAL`	The usual durability level for WAL.
`foreign_keys=ON`	Foreign keys are enforced by SQLite.

`core/tests/test_core.py::test_sqlite_runs_in_wal_mode_with_immediate_transactions` checks this.

Limitations. One writer at a time; all workers must run on the host that holds the file; no network access to the database; `backup_db` refuses any other engine. The settings are generic Django, so a move to PostgreSQL would be a settings change plus data migration, but that path is **not implemented or tested**.

1.2 Data model in one page

Twelve CodeGuard models in five apps (13 tables, because `SecurityControl.stages` adds a join table), on top of Django's `auth.User`.

- **Reference data** (`threats`): `SupplyChainStage` (5 rows, seeded), `SecurityControl` (14 rows, seeded) with an M2M to stages.
- **Scenarios** (`threats`): `ThreatScenario` → `PropagationRule` (edges) and `ScenarioControl` (relevant controls). Six system scenarios are seeded.
- **Runs** (`simulation`): `SimulationRun` (UUID key) → `SimulationStageResult`, `PropagationEvent`.
- **Follow-up** (`remediation`): `RemediationItem`.
- **People** (`accounts`): `Profile` (role), `UserPreference`.
- **History** (`audit`): `AuditEvent`.

Three modelling choices are worth understanding before reading the migrations:

1. **Integrity in the schema, not only in forms.** Ranges, uniqueness and the "no self-loop" rule are `CheckConstraint`/`UniqueConstraint` objects in `Meta.constraints`, so the database refuses bad rows even if a code path skips form validation (`threats/tests/test_scenarios.py::TestConstraints`).

2. **Denormalised history.** Runs copy `scenario_name`, `scenario_slug`, `scenario_version` and store stage keys/names as text, so `on_delete=SET_NULL` on `scenario` loses nothing.

3. **A partial unique constraint.** `RemediationItem` allows only one *active* item per owner, scenario and control:

   ```
   models.UniqueConstraint(
       fields=["owner", "scenario_slug", "control"],
       condition=Q(status__in=["open", "in_progress"]),
       name="one_active_item_per_control_and_scenario",
       ...
   )
   ```

   Verified SQL (`sqlmigrate remediation 0001`):

   ```
   CREATE UNIQUE INDEX "one_active_item_per_control_and_scenario"
     ON "remediation_remediationitem" ("owner_id", "scenario_slug", "control_id")
     WHERE "status" IN ('open', 'in_progress');
   ```

1.3 JSON columns

`SimulationRun.enabled_controls`, `input_snapshot`, `result`, `PropagationEvent.controls_applied` and `AuditEvent.metadata` are `JSONField`s (stored as text in SQLite, queried with Django's JSON lookups, e.g. `metadata__attempted_username__iexact` in `accounts/throttling.py`). The snapshot carries `snapshot_version` (currently 2; version 2 added `control_assumptions`), and the code comment in `simulation/services.py` says readers "must tolerate its absence in version-1 snapshots". Schema changes inside JSON are therefore versioned by hand, not by migrations.

1.4 Append-only audit table

`AuditEvent` refuses changes in Python: `save()` raises when `pk` is set, `delete()` raises, and its manager is `AppendOnlyQuerySet`, whose `update()` and `delete()` raise. The only sanctioned delete is `purge_audit_events`, which calls the private `_purge()`. SQLite itself would accept an `UPDATE`: there are no triggers (`docs/engineering-assessment.md` H3, "database-level triggers remain future work").

1.5 Backup and restore

`python manage.py backup_db [--output DIR] [--keep N]` (`core/management/commands/backup_db.py`) uses SQLite's online backup API (`sqlite3.Connection.backup`), runs `PRAGMA integrity_check` on the copy, deletes it if the check fails, and keeps the newest N copies (default 14) named `codeguard-<UTC timestamp>.sqlite3`. Restore is a documented manual procedure in `docs/deployment.md` §7. Status: the command is covered by `test_backup_db_writes_a_verified_copy_and_rotates`; a restore drill is not automated.

Part 2 — Django migrations, taught with CodeGuard's files

2.1 What a migration is

A migration is a Python file that describes one step in the evolution of the database schema (and optionally data). Django keeps, for each app, an ordered chain of these files in `<app>/migrations/`. The database records which ones have been applied in the table `django_migrations` (columns `app`, `name`, `applied`).

Why they exist. Models change over time. Migrations let every copy of the database (a developer laptop, a CI database, production) be brought to the same schema by running the same files in the same order, and they preserve existing rows while doing it.

How it works, in four steps:

1. You change `models.py`.
2. `python manage.py makemigrations` compares the models with the state described by the existing migration files and writes a new file containing the difference as *operations*.
3. `python manage.py migrate` applies every unapplied migration, in dependency order, inside a transaction (SQLite supports transactional DDL), and inserts a row into `django_migrations`.
4. Commit the migration file together with the model change.

Django never inspects the live database to decide what to generate: it replays the migration files in memory to build the "project state". That is why a missing migration file is a bug and why CI runs `makemigrations --check --dry-run`.

2.2 Anatomy of a migration: `accounts/migrations/0001_initial.py`

```
class Migration(migrations.Migration):

    initial = True

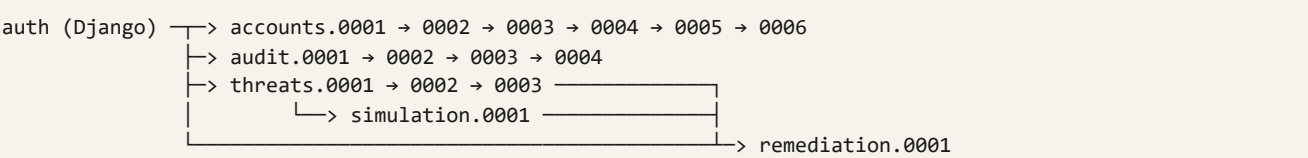
    dependencies = [
        migrations.swappable_dependency(settings.AUTH_USER_MODEL),
    ]

    operations = [
        migrations.CreateModel(
            name='Profile',
            fields=[
                ('id', models.BigAutoField(auto_created=True, primary_key=True, serialize=False,
verbose_name='ID')),
                ('role', models.CharField(choices=[('admin', 'Administrator'), ('analyst', 'Analyst'),
('viewer', 'Viewer')], default='viewer', max_length=16)),
                ('created_at', models.DateTimeField(auto_now_add=True)),
                ('updated_at', models.DateTimeField(auto_now=True)),
                ('user', models.OneToOneField(on_delete=django.db.models.deletion.CASCADE,
related_name='profile', to=settings.AUTH_USER_MODEL)),
            ],
            options={
                'constraints': [models.CheckConstraint(condition=models.Q(('role__in', ['admin', 'analyst',
'viewer'])), name='profile_role_valid')],
            },
        ),
    ]
```

- `initial = True` marks the first migration of the app.
- `dependencies` lists migrations that must run first. `swappable_dependency(settings.AUTH_USER_MODEL)` means "whatever migration creates the user model", so the `OneToOneField` has a table to reference.
- `operations` is an ordered list. `CreateModel` becomes `CREATE TABLE`; the `id` field is a `BigAutoField` because `DEFAULT_AUTO_FIELD = "django.db.models.BigAutoField"`.
- The `CheckConstraint` becomes a SQL `CHECK` inside the table definition.
- The header comment (`# Generated by Django 5.2.18 on 2026-10-26 12:12`) shows the file was generated, not hand-written.

2.3 Dependencies across apps

The full graph (from the `dependencies` lists):



`simulation.0001` depends on `threats.0001` because `SimulationRun.scenario` is a foreign key to `ThreatScenario`. `remediation.0001` depends on `simulation.0001` (for `source_run`) and on `threats.0003` (the latest `SecurityControl` state when it was generated). Django orders everything topologically, so `migrate` on an empty database works without you choosing an order. **Verified:** a fresh `migrate` applied all files, ending with `simulation.0001_initial… OK`, `remediation.0001_initial… OK`, `sessions.0001_initial… OK`.

2.4 Adding fields: `accounts/migrations/0002_preferences.py` and `threats/0003`

`0002_preferences` contains three operations in one file: two `AddField` operations on `profile` (`job_title`, `team`, both `blank=True`) and a `CreateModel` for `UserPreference`. Because both new `CharField`s default to empty strings, existing profile rows get `''` and no question is asked.

`threats/migrations/0003_control_assumptions.py` adds two `TextField(blank=True)` columns to `SecurityControl`. `sqlmigrate` shows what SQLite actually does:

```
-- Add field effectiveness_basis to securitycontrol
CREATE TABLE "new__threats_securitycontrol" (... "effectiveness_basis" text NOT NULL, ...
    CONSTRAINT "control_effectiveness_range" CHECK ("effectiveness" <= 100));
INSERT INTO "new__threats_securitycontrol" (..., "effectiveness_basis")
    SELECT ..., '' FROM "threats_securitycontrol";
DROP TABLE "threats_securitycontrol";
ALTER TABLE "new__threats_securitycontrol" RENAME TO "threats_securitycontrol";
```

Lesson: SQLite's `ALTER TABLE` is limited, so Django rebuilds the table: create a new one, copy the rows (filling `''` for the new column), drop the old one, rename. This happens twice, once per added field. It is safe inside a transaction but costs time proportional to table size, which is negligible for 14 control rows.

2.5 Changing only `choices`: the surface migrations and the audit actions

`0003_surface_preference` added `surface` with choices `paper`, `neutral`. `0004_pearl_surface` and `0005_sage_surface` each widened the choices with an `AlterField`. Verified:

```
$ python manage.py sqlmigrate accounts 0004
BEGIN;
-- Alter field surface on userpreference
-- (no-op)
COMMIT;
```

Choices are enforced by Django forms and `full_clean()`, not by the database, so the SQL is a no-op. The migration still has to exist because the choices are part of the field definition in the project state. The same pattern drives `audit.0002`–`0004`: each new `AuditAction` member required an `AlterField` on `AuditEvent.action` (25 choices today), and each one is a SQL no-op.

Lesson: forgetting such a migration does not break the database, but `makemigrations --check` (run by `tests/test_integration.py::test_migrations_are_consistent_with_models`, GitHub Actions and the Jenkinsfile) fails the build.

2.6 A data migration: `accounts/migrations/0006_retire_neutral_surface.py`

When the "Neutral" surface was retired, existing users could still hold that value. The migration moves them to the default before narrowing the choices:

```
def move_neutral_to_paper(apps, schema_editor):
    """The Neutral surface was retired; its users get the default, Paper."""
    UserPreference = apps.get_model("accounts", "UserPreference")
    UserPreference.objects.filter(surface="neutral").update(surface="paper")


class Migration(migrations.Migration):
    dependencies = [("accounts", "0005_sage_surface")]

    operations = [
        # Data first, so no row is left holding a value the new choices reject.
        # Reversing does nothing: there is no record of who had chosen Neutral.
        migrations.RunPython(move_neutral_to_paper, migrations.RunPython.noop),
        migrations.AlterField(
            model_name="userpreference",
            name="surface",
            field=models.CharField(choices=[("paper", "Paper"), ("pearl", "Pearl"), ("sage", "Sage")],
default="paper", max_length=8),
        ),
    ]
```

Four rules this file demonstrates:

1. **Use `apps.get_model`, never import the model.** The function receives the *historical* model as it was at this point of the chain. Importing `accounts.models.UserPreference` would give today's model, which may have fields that do not exist yet when an old database is migrated.
2. **Order the operations.** Data first, then the schema/choice change.
3. **Decide what reversing means.** `RunPython.noop` makes the migration reversible without pretending to restore lost information. **Verified:** `migrate accounts 0005` printed `Unapplying accounts.0006_retire_neutral_surface... OK`, `showmigrations` then showed `[ ] 0006_retire_neutral_surface`, and `migrate accounts` re-applied it.
4. **Test the behaviour.** `accounts/tests/test_settings.py::test_retired_neutral_surface_is_rejected` checks that the old value is no longer accepted.

`sqlmigrate accounts 0006` prints `THIS OPERATION CANNOT BE WRITTEN AS SQL` for the `RunPython` step: Python operations only exist at run time.

2.7 Changing a constraint's message: `threats/0002`

`0002_friendly_rule_constraint_messages` uses `AlterConstraint` (new in Django 5.2's migration operations) to add `violation_error_message` to `unique_rule_per_edge` and `rule_not_self_loop`. `sqlmigrate threats 0002` shows `(no-op)` twice: the message is used by Django's model validation (`validate_constraints`), not by SQLite.

2.8 Commands you will use

Task	Command (from the project root, venv active)
Apply everything	`python manage.py migrate`
See what is applied	`python manage.py showmigrations [app]`
Generate after a model change	`python manage.py makemigrations [app]`
Check nothing is missing (CI)	`python manage.py makemigrations --check --dry-run`
Show the SQL of one migration	`python manage.py sqlmigrate <app> <number>`
Roll an app back to a migration	`python manage.py migrate <app> <number>` (e.g. `accounts 0005`)
Roll an app back completely	`python manage.py migrate <app> zero` (drops its tables)

Task	Command (from the project root, venv active)
Write an empty migration for data	`python manage.py makemigrations <app> --empty --name <name>`
Use a throw-away database	set `DJANGO_DB_PATH` to another file first

On Windows in this project, run them as `.venv\Scripts\python.exe manage.py …` because Python is not on `PATH`.

2.9 A safe workflow for a model change (tutorial; not a real feature)

The following illustrates the steps with an invented field; CodeGuard does **not** have it.

1. Edit the model, e.g. add `review_note = models.TextField(blank=True)` to `RemediationItem`.
2. `python manage.py makemigrations remediation` → creates `0002_…py`. Read the file.
3. `python manage.py sqlmigrate remediation 0002` → expect a table rebuild on SQLite.
4. `DJANGO_DB_PATH=<scratch>.sqlite3 python manage.py migrate` → apply to a scratch copy (a restored backup is ideal).
5. `pytest` → the integration test fails if the migration is missing or stale.
6. Commit the model, the migration and any test together.
7. In production: `backup_db`, deploy, `migrate`, as in `docs/deployment.md` §9.

For a **non-nullable field without a default**, `makemigrations` asks for a one-off default for existing rows; for a **rename**, it asks whether the field was renamed (answer yes, or it will drop and re-add the column and lose data); for **removing a choice that rows still use**, write a data migration first, exactly like `accounts.0006`.

2.10 Seed data is not a migration

Reference data (stages, controls, system scenarios) is loaded by `python manage.py seed_codeguard`, not by a data migration. The command is idempotent (`update_or_create` by key/slug), updates stages and controls on every run, leaves existing system scenarios untouched unless `--reset-scenarios` is passed (which increments their version), and creates development-only demo users with `--demo-users`, refusing unless `DJANGO_DEBUG=true`. The Jenkinsfile runs it twice to prove idempotency, and `TestSeedCommand::test_seed_is_idempotent` checks it. **Verified:** on the scratch database it printed `Seeded 5 stages, 14 controls, 6 new system scenarios (6 total).`

Trade-off: a fresh database is empty until the command is run (the README Quick start includes it), but administrators' edits to system scenarios are not overwritten by upgrades.

2.11 Pitfalls observed in this codebase

- Migration headers have two quoting styles (generated vs reformatted); harmless, and Ruff excludes migrations.
- `UserPreference` rows are created lazily (`preferences_for` uses `get_or_create`), so a migration that assumes every user has a preference row would be wrong.
- `Profile` rows are created by a `post_save` signal on the user model; users created before the `accounts` app existed (or by raw SQL) would have none, and `get_role` treats them as viewers.
- Before commit `eef6716`, `templates/reports/` was git-ignored, so a fresh clone migrated fine but failed at render time on the Reports pages: a version-control gap, not a migration problem.

Security

This document describes the security controls **of the CodeGuard application itself** (not the simulated supply-chain controls), what evidence exists for each, and the gaps. The project's own STRIDE analysis is in `docs/threat-model.md`; findings and fixes are in `docs/engineering-assessment.md`. Diagram: authentication.

Status labels: **verified** = implemented and covered by a test that passed on 2026-11-10; **unverified** = implemented, no test found; **partial**; **not implemented**.

No penetration test or formal audit has been done. Passing tests do not prove the absence of vulnerabilities (README says the same).

1. Authentication

What it is. Proving who a user is. CodeGuard uses Django's session authentication with the built-in `User` model and password hashing (PBKDF2 by default in production settings; the test settings switch to MD5 purely for speed).

Control	How	Evidence	Status
Password policy	`AUTH_PASSWORD_VALIDATORS`: similarity, **min length 10**, common-password list, not all numeric	`test_weak_password_is_rejected`	verified
Passwords hashed	Django hasher	`test_password_is_hashed`	verified
Sign-in throttling	`ThrottledAuthenticationForm.clean()` calls `throttling.is_locked()` **before** the password is checked. Defaults: 5 failures per username, 20 per client IP, within 15 minutes. Failures before the account's last successful sign-in are not counted. A locked account stays locked even with the right password	`TestLoginThrottling` (6 tests)	verified
Failed attempts recorded without passwords	`signals.on_login_failed` stores only `attempted_username` (truncated to 150)	`test_sensitive_metadata_is_removed`, throttling tests	verified
Password reset	Django `PasswordResetView`; email via console backend by default, SMTP configurable; response does not reveal whether an account exists	`test_password_reset_sends_email_without_disclosing_accounts`	verified
Password change audited	`AuditedPasswordChangeView.form_valid` → `password_change` event	`test_password_change_is_audited`	verified
Email change needs the current password	`settings_views.account`	`test_email_change_requires_the_current_password`	verified
Logout requires POST	Django 5 `LogoutView`	`test_logout_requires_post_and_ends_session`	verified

Control	How	Evidence	Status
Open-redirect protection on login	Django's `next` validation	`test_open_redirect_is_ignored`	verified
Multi-factor authentication for CodeGuard accounts	—	—	**not implemented** (MFA appears only as a *simulated* control)

Throttling trade-off (from the docstring of `accounts/throttling.py`). Anyone can lock an account for 15 minutes by guessing wrongly five times. Counting is done from audit events in the database, so it works across worker processes without a cache server.

Gap: client IP behind a proxy. `audit.services.client_ip` deliberately trusts only `REMOTE_ADDR` ("X-Forwarded-For is client-controlled unless a trusted proxy is configured"). Behind a reverse proxy every request has the proxy's address, so the per-IP limit (20) becomes a limit shared by **all** users, and audit events record the proxy IP. `docs/deployment.md` mentions raising the IP limit for shared NAT addresses but not this proxy effect. Status: **partial**.

2. Sessions

Control	Setting / code	Status
HttpOnly session cookie	`SESSION_COOKIE_HTTPONLY = True`	configured (Django default behaviour, not separately tested)
SameSite=Lax cookies	`SESSION_COOKIE_SAMESITE`, `CSRF_COOKIE_SAMESITE`	configured
8-hour lifetime, ends at browser close	`SESSION_COOKIE_AGE = 28800`, `SESSION_EXPIRE_AT_BROWSER_CLOSE = True`	configured
Secure cookies when DEBUG is off	`SESSION_COOKIE_SECURE`/`CSRF_COOKIE_SECURE` default to `not DEBUG`	covered by `check --deploy`
"Sign out of other sessions"	`UserPreference.session_epoch`; `revoke_sessions` increments it and stores the new value in the current session; `SessionEpochMiddleware` logs out any session whose stored epoch differs	`test_signing_out_other_sessions` — verified
Session list	`/settings/sessions/` shows the current session (start time, a short user-agent description from `describe_agent`) and recent sign-ins from the audit log; Django's database sessions are not enumerated	verified (`test_sessions_page_describes_this_session`)

3. Authorization

What it is. Deciding what an authenticated user may do. CodeGuard has three roles stored in `accounts_profile.role` and resolved by `accounts.permissions.get_role` (superusers are always admin; a missing profile means viewer). Templates hide buttons for convenience, but the module docstring is explicit: "every protected view calls these helpers… so authorization never depends on what the browser shows."

3.1 Role matrix (from decorators and service checks)

Capability	Viewer	Analyst	Admin
Read shared scenarios, controls, stages, help	✓	✓	✓
Read own private scenarios	— (cannot create)	✓	✓
Create / clone scenarios (always private)	✗	✓	✓ (may publish as shared)
Edit / archive a scenario	✗	own private only	own private + all shared
Delete a scenario	✗	own private, never system	same; system scenarios never
Run simulations	✗ (`analyst_required`)	✓	✓
View reports	own + shared completed	own + shared completed	own + shared completed (no special access to private runs)
Share / unshare a report	—	owner only	owner only
Add to / update the remediation plan	✗	own items	own items
Audit log	own events	own events	all events, filter by user
Users and roles, organization page	✗	✗	✓ (cannot change own role or a superuser's)

Evidence: `TestRoles`, `TestScenarioPermissions`, `TestConfigureView::test_viewer_cannot_run_simulations`, `TestReportAccess`, `test_viewers_cannot_track_and_strangers_cannot_reach_private_runs`, `test_administration_pages_are_admin_only`, `test_admin_cannot_see_other_users_private_scenarios`. Status: **verified**.

3.2 Object-level access (IDOR protection)

All lookups go through scoped querysets, and misses return **404, not 403**, so the existence of another user's object is not revealed:

- `threats.services.visible_scenarios(user)`: `Q(visibility="shared") | Q(owner=user)`.
- `simulation.services.accessible_runs(user)`: `Q(user=user) | Q(is_shared=True, status="completed")`; `get_accessible_run` also catches malformed UUIDs (`ValueError`).
- `remediation.services.items_for(user)`: `owner=user`.
- `SimulationForm.__init__` limits the scenario choice to *active, visible* scenarios, so "a forged POST cannot run another user's private scenario" (`test_cannot_run_another_users_private_scenario`).
- `test_viewer_role_denial_does_not_reveal_whether_a_private…` checks that the role denial happens before the lookup.

A live HTTP authorization matrix (`tools/authz_live.py`, run as a second analyst, a viewer and anonymously, with real CSRF tokens) passed according to the README; it was **not re-run** for this documentation.

3.3 Registration and the tenancy boundary

`RegistrationForm` has no role field; `signals.create_profile` assigns `CODEGUARD_DEFAULT_ROLE` (only `analyst` or `viewer` are accepted by settings). Who may register is set by `CODEGUARD_REGISTRATION`: `open` (default), `domain` (with `CODEGUARD_REGISTRATION_DOMAINS`) or `closed`. With DEBUG off and registration open, `manage.py check --deploy`

warns `codeguard.W001`. Verified by `TestRegistrationPolicy` and by running `scripts/ci_deploy_check.py` (warning present).

Gap (documented, by design). "Shared" means every account on the instance; there are no organizations. With the default `open` registration on a public host, a stranger can register as an analyst and read every shared scenario and report. Status: single-tenant model **implemented**; multi-tenant organizations **not implemented** (assessment H2).

4. Request integrity and browser hardening

Control	Implementation	Evidence	Status
CSRF	`CsrfViewMiddleware`; every form has `{% csrf_token %}`; state changes use POST (`require_POST` on share, archive, clone, track, revoke, role change, notifications read, theme)	`test_login_form_enforces_csrf`, `test_archive_requires_post`, `test_csrf_failure_page`	verified
Content-Security-Policy	`core/middleware.py`: `default-src 'self'`; `script-src 'self'` plus the SHA-256 hash of the one inline theme script (`core/theme.py`); styles and fonts from self and Google Fonts; `img-src 'self' data:`; `form-action 'self'`; `frame-ancestors 'none'`; `object-src 'none'`; `base-uri 'self'`	`test_security_headers_present`, `test_theme_script_runs_first_and_is_the_only_inline_script_the_csp_allows`	verified
Clickjacking	`X_FRAME_OPTIONS = "DENY"` plus `frame-ancestors 'none'`	header test	verified
Other headers	`nosniff`, `Referrer-Policy: same-origin`, `Cross-Origin-Opener-Policy: same-origin`, `Permissions-Policy: camera=(), microphone=(), geolocation=()`	header test	verified
HTTPS	`SECURE_SSL_REDIRECT`, HSTS 3600 s by default when DEBUG is off; `/healthz/` exempt from the redirect	`check --deploy`	configured; HTTPS itself not exercised (no TLS server here)
Template auto-escaping	Django default. `mark_safe` appears only for fixed markup in `core/templatetags/codeguard_tags.py` (theme script, level ticks); the `safe` filter only on developer-defined form `help_text` (`components/field.html`, `registration/_simple_form.html`)	—	unverified (no dedicated XSS test)
CSV formula injection	`simulation/exports.safe_cell` prefixes `'` to cells starting with `= + - @ \t \r`	`test_csv_cells_are_protected_against_formula_injection`	verified
Safe redirects	`quick_theme` validates `next` with `url_has_allowed_host_and_scheme`	`test_quick_theme_toggle_saves_the_color_mode`	verified
Error pages	custom 400/403/404/500 and CSRF pages; no stack traces when DEBUG is off	`test_custom_404_page_has_no_debug_details`, `test_server_error_page_hides_details`	verified
Health endpoint discloses nothing	returns only `status` and `database`	`test_health_check_is_public_and_minimal`	verified

Exclusions. The CSP header is not sent for `/admin/` (the middleware skips it), and the Django admin's own changes are not written to the CodeGuard audit trail. `docs/deployment.md` suggests restricting `/admin/` at the proxy. Status: **partial**.

5. Secrets and configuration

- `DJANGO_SECRET_KEY` is required when DEBUG is off (`ImproperlyConfigured` otherwise). A fixed development fallback exists only when DEBUG is true.
- `.env` is git-ignored (`.env`, `.env.*`, with `!.env.example`); `.env.example` contains only placeholders. The local `.env` was not opened for this documentation.
- `scripts/ci_deploy_check.py` generates a random throw-away key per process and never prints it.
- The demo accounts' password is published in the README on purpose. The seed command refuses `--demo-users` unless DEBUG is true (`test_demo_users_refused_outside_debug`), but a database seeded in development and then deployed would still contain them. Status: **partial** (no check warns about demo users in production).
- Ruff runs the Bandit-derived `S` rules on all code; test passwords are allow-listed per file in `pyproject.toml`.

6. Audit trail and privacy

`audit.services.record_event` is called for sign-in, sign-out, failed and throttled sign-in, registration, password/email change, scenario create/update/archive/restore/delete/clone, simulation run/failure, report share/export, role change, audit purge, remediation create/update, profile and preference updates, session revocation and personal data export (the 25 `AuditAction` values). It stores actor, username, action, object type/id/repr, sanitised metadata and `REMOTE_ADDR`.

- `sanitize_metadata` drops keys containing `password`, `secret`, `token`, `session`, `csrf`, `key` or `credential`, and truncates values to 200 characters and lists to 50 items.
- Append-only at the ORM level (see `DATABASE_AND_MIGRATIONS.md` §1.4); deleting a user keeps their events with `actor = NULL`.
- Retention: `purge_audit_events --days N` (minimum 30, `--dry-run`, records its own run).
- Personal data export: `/settings/privacy/export.json` returns only the user's own data and is audited (`test_data_export_contains_only_own_data_and_is_audited`).

Gaps: no database-level immutability (triggers), no tamper-evident chaining, no off-host log shipping; IP addresses are personal data and `docs/deployment.md` §8 says to cover them in a privacy notice.

7. Supply chain of CodeGuard itself

All Python dependencies are pinned in `requirements.txt` and `requirements-dev.txt`; `scripts/ci_dependency_audit.py` runs `pip-audit` with explicit exit codes (0 clean, 1 vulnerable, 2 incomplete). It reported "No known vulnerabilities" when re-run on 2026-11-10, and the same check passes in GitHub Actions (run #7). Bootstrap and Bootstrap Icons are vendored (no CDN), but their versions are not recorded in the repository. Hashes (`--require-hashes`) are not used.

8. Summary of gaps

Gap	Severity (author's judgement)	Status
Reports templates were not in Git: a deployment from the repository served 500 errors on `/reports/`	high (availability)	fixed in commit `eef6716`

Gap	Severity (author's judgement)	Status
No MFA for CodeGuard accounts	medium	not implemented
Per-IP throttling collapses to one shared bucket behind a proxy	medium	open
Open registration by default, single-tenant sharing	medium	documented; mitigated by settings and `W001`
Admin outside CSP and audit trail	low	documented
Audit immutability only in the ORM	low	documented
Demo users could survive into production	low	partial
Unpinned `requests` for `tools/authz_live.py`; vendored asset versions unknown	low	open
No penetration test, no screen-reader test	—	not done

Simulation and risk methodology

This document states exactly what the CodeGuard engine computes, using the source as the only authority. Every formula below is quoted from or traceable to `simulation/risk.py`, `simulation/propagation.py`, `simulation/engine.py` and `simulation/recommendations.py` (`MODEL_VERSION = "1.0"`). The worked example was produced by running the real engine on the seeded data on 2026-11-10 and was then recomputed by hand.

Status: implemented and verified. 91 unit tests target these modules (`test_risk.py` 44, `test_engine.py` 41, `test_recommendations.py` 6) and all pass; branch coverage of the four modules is 99 %.

What it is not. The model is a deterministic teaching model. It uses no telemetry and inspects no real system. Its probabilities and control effectiveness values are illustrative parameters, not measurements (`engine.LIMITATIONS`, `threats/seed_data.py` header).

1. Inputs

The engine consumes one immutable `SimulationInput` (`simulation/domain.py`). Every field comes from the database through `simulation.services.build_input`, or from a stored snapshot through `input_from_snapshot`.

Symbol	Field	Range	Source in the app
S	`stages`	≥ 1 stage; unique `key` and `position`; integer `weight` ≥ 1	all `SupplyChainStage` rows, ordered by `position`
e	`entry_stage`	a key in S	`ThreatScenario.initial_stage`
L_0	`likelihood`	0–100	form field "Base likelihood (%)", default from the scenario
σ	`severity`	integer 1–5	form field, default from the scenario
ι	`impact`	integer 1–5	form field, default from the scenario
E	`edges`	each `Edge(source, target, probability p)` with p in 0–100; no self-loops, no duplicates, both ends in S	the scenario's `PropagationRule` rows
C	`controls`	each `Control(key, effectiveness ε, stages, category, …)`, ε in 0–100, keys unique, stages in S	the controls ticked on the form
τ	`threshold`	> 0 and ≤ 100 (form: integer 1–100)	form field, default `propagation_threshold` (50)

`engine.validate_input` rejects anything outside these ranges with a `SimulationConfigError` before a run row is created (`services.run_simulation` calls it first). Booleans are not accepted as numbers (`risk._is_number`), and severity/impact must be real integers (`validate_level`).

A control acts in one of two **modes**, decided by its category (`domain.CATEGORY_MODES`):

```
CATEGORY_MODES = {
    "preventive": MODE_PREVENT,
    "detective": MODE_MITIGATE,
    "recovery": MODE_MITIGATE,
}
```

- **prevent**: lowers the probability that a compromise *enters* a stage the control protects;

- **mitigate**: does not change propagation, but lowers the *residual risk* of a protected stage.

2. Formulas

2.1 Combined control coverage

For each stage s and each mode m, the coverage is the combined effectiveness of all enabled controls of that mode that list s among their stages (`engine._coverage_by_stage`, `risk.combined_effectiveness`):

```
coverage_m(s) = 100 × (1 − ∏ (1 − εᵢ / 100))      over controls i with mode m protecting s
```

With no applicable controls the product is empty (= 1) and coverage is 0. Because each factor is in [0, 1], coverage can approach but never exceed 100. The assumption, stated in `LIMITATIONS`, is that controls act independently.

Write P(s) = coverage_prevent(s) and M(s) = coverage_mitigate(s).

2.2 Edge evaluation

For an edge u → v with probability p (`propagation.traverse`):

```
effective(u→v) = p × (1 − P(v) / 100)
```

The outcome is decided with `risk.meets_threshold`, which compares values rounded to 6 decimal places (`DECISION_PRECISION = 6`) so that 49.9999999 cannot flip a decision:

Condition	Outcome constant	Meaning
effective ≥ τ	`propagated`	the compromise spreads to v
effective < τ and p ≥ τ	`blocked_by_controls`	it would have spread without controls
p < τ	`below_threshold`	it would not have spread anyway

The comparison is inclusive: an edge exactly at the threshold propagates (`test_threshold_boundary_is_inclusive`).

2.3 Traversal order

Breadth-first from e. `compromised = {e}`. Each time a stage leaves the queue, **all** its outgoing edges are evaluated, in order of target position. A newly compromised target is queued; an already compromised one is recorded (`target_already_compromised=True`) but never re-queued, so every edge is evaluated at most once and cycles terminate. Edges whose source is never compromised are never evaluated; the pipeline diagram shows them as "Not evaluated".

2.4 Stage likelihood

For compromised stages, the likelihood is the most likely successful path (`propagation._path_likelihoods`, a max-product Dijkstra variant over `propagated` edges only):

```
λ(e) = L_entry
λ(v) = max over successful paths e → … → v of  L_entry × ∏ (effective_k / 100)
```

Since each factor is ≤ 1, a longer path can never raise the value, which keeps the search well defined on cyclic graphs. Paths are not summed (stated in `LIMITATIONS`).

Every evaluated edge then gets a **transmitted likelihood**:

```
transmitted(u→v) = λ(u) × effective(u→v) / 100
```

A stage that was attempted but not compromised keeps the likelihood of its strongest failed attempt, its "residual exposure":

```
λ(v) = max over evaluated edges into v of transmitted(u→v)      if v is not compromised
λ(v) = 0                                                        if no edge into v was evaluated
```

2.5 Stage state

From `traverse` (labels from `engine.STATE_LABELS`):

State	Rule
`entry`	v = e
`compromised`	v was reached by a propagated edge
`blocked`	not compromised, and at least one incoming evaluation is `blocked_by_controls`
`exposed`	not compromised, has incoming evaluations, none of them blocked
`unaffected`	no incoming evaluation

2.6 Two passes: inherent and residual

`engine.simulate` runs the traversal twice:

- **inherent pass**: `traverse(..., L₀, coverage={}, ...)`, i.e. no controls at all;
- **residual pass**: `traverse(..., L_entry, coverage=P, ...)` with

```
L_entry = L₀ × (1 − P(e) / 100)
```

so preventive controls on the entry stage reduce the starting likelihood.

2.7 Stage risk

`risk.raw_risk` and `risk.residual_risk`:

```
raw(λ)          = λ × (σ / 5) × (ι / 5)                  # 0–100
inherent_risk(s) = raw(λ_inherent(s))
residual_risk(s) = raw(λ_residual(s)) × (1 − M(s) / 100)
```

Mitigating coverage is applied only to the residual value; the inherent value ignores every control by construction.

2.8 Overall risk

The criticality-weighted average over **all** stages, including stages with zero risk (`risk.weighted_overall`):

```
overall = Σ (risk(s) × w(s)) / Σ w(s)
```

The seeded weights are IDE 1, Version Control 2, CI/CD 2, Registry 2, Production 3 (Σw = 10). The weighted average uses the unrounded stage values (`exact_inherent`, `exact_residual` in `engine.simulate`); only the result is rounded.

2.9 Rounding and classification

`risk.round_score` clamps to 0–100 and rounds half-up to one decimal place using `Decimal` (so 0.05 → 0.1, not banker's rounding). Every stored and displayed score is rounded this way. Classification (`risk.classify`) uses inclusive lower bounds:

Band	Score
critical	≥ 50
high	≥ 30 and < 50
moderate	≥ 15 and < 30
low	< 15

Stage classifications use the rounded residual risk; the run's `classification` uses the rounded overall residual risk. `peak_inherent_risk` and `peak_residual_risk` are the maxima of the rounded stage values.

2.10 Counters

`compromised_count`: stages in `entry` or `compromised` in the residual pass. `inherent_compromised_count`: the same in the inherent pass. `blocked_count`: residual evaluations with outcome `blocked_by_controls`. `contained_count`: residual evaluations with outcome `below_threshold`.

3. Worked example

Scenario: the seeded "Compromised Developer Account" (`threats/seed_data.py`): entry stage IDE, $L_0 = 70$, $\sigma = 4$, $\iota = 4$, $\tau = 50$. Rules:

Edge	p	Kind
IDE → Version Control	85	forward
IDE → CI/CD	40	bypass
Version Control → CI/CD	80	forward
CI/CD → Registry	70	forward
Registry → Production	65	forward

Enabled controls: `mfa` (preventive, 60, protects IDE and Version Control), `protected-branches` (preventive, 50, Version Control), `sca` (detective, 40, CI/CD and Registry).

The severity-impact factor is (4/5) × (4/5) = 0.64 throughout.

Step 1: coverage

Stage	Preventive P	Mitigating M
IDE	MFA alone: 60	0
Version Control	100 × (1 − 0.4 × 0.5) = 80	0
CI/CD	0	SCA: 40

Stage	Preventive P	Mitigating M
Registry	0	SCA: 40
Production	0	0

Step 2: inherent pass (no controls)

Order	Edge	effective	Outcome	Target likelihood
1	IDE → VCS	85	propagated	70 × 0.85 = 59.5
2	IDE → CI/CD	40	below threshold	(path through VCS is better)
3	VCS → CI/CD	80	propagated	59.5 × 0.80 = 47.6
4	CI/CD → Registry	70	propagated	47.6 × 0.70 = 33.32
5	Registry → Production	65	propagated	33.32 × 0.65 = 21.658

Inherent stage risk = λ × 0.64: IDE 44.8, VCS 38.08, CI/CD 30.464, Registry 21.3248, Production 13.8611. All five stages are compromised.

```
inherent overall = (44.8×1 + 38.08×2 + 30.464×2 + 21.3248×2 + 13.8611×3) / 10
                 = 266.121 / 10 = 26.6121  →  26.6 (moderate)
```

Step 3: residual pass

L_entry = 70 × (1 − 60/100) = **28.0**.

Order	Edge	effective	Outcome	transmitted
1	IDE → VCS	85 × (1 − 0.80) = 17.0	17 < 50 but 85 ≥ 50 → **blocked by controls**	28 × 0.17 = 4.76
2	IDE → CI/CD	40 × (1 − 0) = 40.0	40 < 50 and 40 < 50 → **below threshold**	28 × 0.40 = 11.2

Nothing propagated, so the queue empties after IDE and the three VCS/CI/CD/Registry edges are never evaluated.

Stage	State	λ residual	raw = λ × 0.64	× (1 − M/100)	residual risk
IDE	entry	28.0	17.92	× 1.0	**17.9** (moderate)
Version Control	blocked	4.76	3.0464	× 1.0	**3.0** (low)
CI/CD	exposed	11.2	7.168	× 0.6	**4.3** (low)
Registry	unaffected	0	0	× 0.6	**0.0** (low)
Production	unaffected	0	0	× 1.0	**0.0** (low)

```
residual overall = (17.92×1 + 3.0464×2 + 4.3008×2 + 0×2 + 0×3) / 10
                 = 32.6144 / 10 = 3.26144  →  3.3 (low)
```

Engine output (verbatim from `SimulationResult.explanation`):

> The attack starts at IDE / Developer Environment. With the enabled controls, 1 of 5 stages are modeled as compromised (IDE / Developer Environment). Overall residual risk is 3.3 (low), compared with an inherent risk of 26.6 (moderate) when no controls are applied. Controls reduce the number of compromised stages from 5

> to 1. 1 propagation attempt(s) that would otherwise succeed were blocked by controls. 1 propagation attempt(s) fell below the 50% threshold without needing controls. The highest remaining stage risk is IDE / Developer Environment at 17.9.

Counters: `compromised_count` 1, `inherent_compromised_count` 5, `blocked_count` 1, `contained_count` 1.

Two details that the example makes visible:

1. A **blocked** stage still carries risk (VCS 3.0) because its likelihood is the transmitted likelihood of the failed attempt. Blocking is not the same as zero risk.
2. SCA on the Registry has no effect here: the Registry was never reached, so there is no risk for it to mitigate. That is also why `recommend` skips controls whose stages all have zero residual risk.

4. Recommendations

`recommendations.recommend(inp, result, catalog, applicable_keys, limit=5)`:

1. Candidates are catalogue controls that are **not enabled** and protect **at least one stage whose residual risk is > 0**.
2. For each candidate the engine is re-run with the control added (`simulate_with_controls`).
3. `risk_reduction = round_score(current overall − trial overall)`; candidates with a reduction ≤ 0 are dropped.
4. Sorting key: largest reduction first, then controls the scenario lists as applicable (`ScenarioControl`), then lower effort (`low < medium < high`), then higher priority, then name.
5. The top five are kept. Each carries a generated `reason`, the protected stages, the edges it changes (for preventive controls) and the stages that would no longer be compromised.

For the worked example the stored recommendations are:

Rank	Control	Reduction	Projected residual	Listed for the scenario?
1	`dependency-pinning` (preventive, 50, IDE + CI/CD)	1.9	1.4	no
2	`incident-response` (recovery, 25, all stages)	0.9	2.4	yes
3	`least-privilege` (preventive, 35, VCS…Production)	0.6	2.7	yes
4	`cicd-secrets` (preventive, 50, CI/CD)	0.5	2.8	no
5	`isolated-builds` (preventive, 45, CI/CD)	0.4	2.9	no

Dependency pinning wins because it also protects the entry stage: P(IDE) becomes 100 × (1 − 0.4 × 0.5) = 80, so L_entry falls from 28 to 14.

5. Control contributions (leave-one-out)

`recommendations.control_contributions` re-runs the engine once per enabled control **without** that control and reports `round_score(risk_without − risk_with)`:

Control	Residual without it	Contribution
`mfa`	10.4	7.1
`protected-branches`	3.9	0.6
`sca`	3.8	0.5

Contributions do not add up to the total reduction (26.6 − 3.3 = 23.3) because controls interact: without MFA, protected branches alone still block IDE → VCS (85 × 0.5 = 42.5 < 50). This is expected for a non-linear model and is why the report shows both views.

6. Determinism and reproducibility

- The engine has no randomness and no clock (Ruff rule `S311` is ignored project-wide with the comment "no cryptographic randomness used in the engine"; there is no `random` import in it).
- Outgoing edges are sorted by target position and controls by slug (`build_input`), so the order of rows in the database does not change the result (`test_edge_order_does_not_change_result`).
- Each run stores `input_snapshot` (snapshot version 2: scenario metadata, rules, applicable controls, control assumptions, and the exact engine input) and `result` (the full serialised `SimulationResult` plus recommendations and contributions). `input_from_snapshot` rebuilds the input; `test_snapshot_reproduces_the_stored_result` proves that re-running it gives the stored result.

7. Limitations (from `engine.LIMITATIONS`, plus observations)

Stated in the code and shown on every report's "Inputs and method" tab and on `/help/`:

1. Propagation uses a fixed threshold on modelled probabilities; it is a simplified model of conditional likelihood, not a calibrated prediction.
2. Control effectiveness values are illustrative teaching parameters.
3. Controls are assumed independent: combined coverage is $1 - \prod(1 - \varepsilon)$.
4. Stage likelihood follows the single most likely path; several paths are not summed.
5. No real-world telemetry; no real system is inspected.

Observed while documenting (not stated in the code):

6. The threshold makes outcomes discontinuous: moving an edge from 49 % to 50 % can compromise every downstream stage at once.
7. Severity and impact are per scenario, not per stage; only the weights differ between stages.
8. Overall risk is an average over all five stages, so an attack that stays in one heavily weighted stage can score lower than intuition suggests.
9. There is no uncertainty range (no Monte-Carlo mode); `docs/engineering-assessment.md` lists this as M8, future work.

Testing and quality assurance

This document separates three things that are easy to blur: **tests that exist**, **tests that were run for this documentation (2026-11-10)**, and **checks that were run earlier and only recorded** in the README or `docs/test-plan.md`. Inventory: tests.

1. What was actually run on 2026-11-10

Check	Command	Result
Full suite, working tree	`.venv\Scripts\python.exe -m pytest`	**337 passed** (36 s)
Coverage	`pytest --cov`	**96 %** total (branch coverage, `pyproject.toml` `[tool.coverage]`)
Core gate	`coverage report --include=simulation/engine.py,simulation/risk.py,simulation/propagation.py,simulation/recommendations.py --fail-under=85`	**99 %**, passed
Full suite on a fresh `git clone` of `8f5d092`	same interpreter	**9 failed, 328 passed** — `templates/reports/*.html` git-ignored
Full suite on a fresh `git clone` of `eef6716` (the fix)	same interpreter	**337 passed**
Lint and format	`ruff check .`; `ruff format --check .`	clean (Markdown excluded from formatting since `8ca8b36`)
Dependency audit	`python scripts/ci_dependency_audit.py`	No known vulnerabilities, exit 0
GitHub Actions	push of `8ca8b36`	**run #7 passed**, every gate, 1 min 5 s
Migrations consistent	`manage.py makemigrations --check --dry-run`	No changes detected
Migrations from scratch	`migrate` on an empty scratch DB, then `seed_codeguard`	all applied; 5 stages, 14 controls, 6 scenarios
Reversible data migration	`migrate accounts 0005`, then `migrate accounts`	unapplied and re-applied `0006`
Deployment check	`python scripts/ci_deploy_check.py`	0 errors, 3 warnings (`codeguard.W001`, `security.W005`, `security.W021`)
Engine worked example	engine called on seeded data	matches the hand calculation in `SIMULATION_AND_RISK_METHODOLOGY.md`

Not run on 2026-11-10: `scripts/rehearse_jenkinsfile.py`, the Node/Chrome checks in `tools/a11y/`, `tools/authz_live.py` (needs a running server and the journey output), and any Jenkins run.

2. Earlier results that are recorded but were not repeated

From the README "Last verified results" table (2026-10-26/28): 332 tests passed at that time; `pip-audit` reported no known vulnerabilities; the Jenkinsfile rehearsal returned SUCCESS; a concurrency test (4 processes × 50 runs) completed 200/200; axe-core reported 0 violations on 47 pages in both themes at 1280 px and 390 px; a keyboard journey passed 14/14 checks in real Chrome; the remediation workflow passed 8/8; the live authorization matrix passed. These are **claims recorded by the project**; this documentation did not reproduce them. The test count has since grown from 332 to 337.

3. Test strategy (as implemented)

Pyramid.

1. **Pure unit tests** of the formulas and the engine (85 tests in `test_risk.py` and `test_engine.py`) use `simulation/tests/factories.py` and never touch the database. They pin every formula with exact expected values, the boundaries of every range, threshold inclusivity, float-noise handling, cycles, bypass and reverse edges, most-likely-path selection, determinism (edge order does not matter) and validation errors.
2. **Service and view tests** with pytest-django's `client`: persistence and rollback, snapshots, access control (IDOR), CSV content and injection, charts and palettes, role checks, form validation, audit events.
3. **Integration tests** (`tests/test_integration.py`): a full analyst journey, every main page rendering for an analyst, every seeded scenario simulating successfully, migrations consistent with models, `manage.py check` clean.
4. **Rule tests on non-Python assets** (`core/tests/test_design.py`): parse `static/css/codeguard.css`, templates and `app.js` to enforce `DESIGN.md` (colours only in token blocks, identical token sets for light and dark, no theme branching, chart token order matches `PALETTE_TOKENS`).
5. **Accessibility tests in Python** (`core/tests/test_accessibility.py`): errors linked to inputs, navigation state, accessible names on the report page.
6. **Optional browser tools** (not part of pytest): `tools/a11y/` (axe-core, keyboard, reflow, journey) and `tools/authz_live.py`.

Isolation. `pytest.ini` uses `codeguard.test_settings`, which forces `DJANGO_DEBUG=false` (so production-mode code paths are tested), a fixed test secret, no HTTPS redirect, the MD5 hasher for speed and the in-memory email backend. pytest-django creates a temporary database per session. `filterwarnings = error::RuntimeWarning` turns runtime warnings (for example naive datetimes) into failures, and `--strict-markers` rejects typos in markers.

Fixtures. Users are created through `conftest._make_user` with explicit roles; `seeded` loads the real seed data so tests use the same scenarios users see.

4. Static quality gates

Gate	Configuration	Where it runs
Ruff lint	`pyproject.toml`: rule sets `E, F, W, I, B, UP, DJ, S`; ignores `E501`, `S101`, `S311`; migrations excluded	`ci.yml`, `Jenkinsfile`, `scripts/ci_local.*`
Ruff format	line length 120	same
Django system checks	`manage.py check`; `check --deploy --fail-level ERROR` via `ci_deploy_check.py`	same
Migration drift	`makemigrations --check --dry-run`	same, plus an integration test
Core coverage gate	≥ 85 % on the four engine modules	same

Gate	Configuration	Where it runs
Dependency audit	`pip-audit -r requirements.txt` with exit codes 0/1/2	same (GitHub Actions treats 2 as failure; Jenkins as UNSTABLE)

5. Requirement-to-test traceability

`docs/test-plan.md` maps requirements (in `docs/requirements.md`) to test names and lists boundary values covered. For this documentation every `test_…` name cited there (94 distinct names) was compared with the collected test list: all exist. Whether each test really proves its requirement was not re-judged.

6. What the tests do not cover

- **Clean checkouts in CI.** Until commit `eef6716` the suite passed only in the developer's working tree (the Reports templates were git-ignored). A fresh clone now passes locally and GitHub Actions passed on run #7; no Jenkins run is on record.
- **Real HTTPS, a real WSGI server and a reverse proxy** (examples in `docs/deployment.md` are marked "not tested here").
- **Concurrency** beyond the one recorded 200-run experiment.
- **Browser behaviour** of `app.js` (tooltips, stepper, theme toggle) except through the optional Node tools.
- **Screen readers** (never tested; README says so).
- **Security testing** beyond functional tests of the controls: no fuzzing, no scanner, no penetration test.
- **Model validity.** The tests prove the engine implements its formulas; nothing can prove the formulas describe real attacks (see the methodology limitations).

7. How to run

```
.venv\Scripts\python.exe -m pytest                      # Windows, this machine
pytest -k engine                                         # a subset by name
pytest simulation/tests/test_risk.py::TestRoundingAndClassification -q
pytest --cov --cov-report=html                           # report in htmlcov/
powershell -ExecutionPolicy Bypass -File scripts\ci_local.ps1   # every gate, Windows
./scripts/ci_local.sh                                    # every gate, Linux/macOS
python scripts/rehearse_jenkinsfile.py                   # Jenkinsfile commands on a fresh clone
```

`scripts/rehearse_jenkinsfile.py` clones HEAD (`checkout()` uses `git clone`), so it tests what is committed, not the working tree; it was not re-run after commit `eef6716`.

Development and deployment

The README's **Quick start** (tested by the project) and `docs/deployment.md` (single-server production guide) already cover installation and operations. This document does not repeat them; it explains how the pieces fit, records the Windows specifics of this machine, describes Git/GitHub and CI status, and lists hosting options honestly.

1. Local development

1.1 Prerequisites

Python **3.12 or newer** (`pyproject.toml`: `requires-python = ">=3.12"`; verified only on 3.12.10), Git, and optionally Node 22+ and Chrome for `tools/a11y/`.

On this development machine Python is installed per user and is **not on** `PATH`: use `.venv\Scripts\python.exe` for every command. Git is at `C:\Program Files\Git\cmd\git.exe`.

1.2 Set-up sequence and why each step exists

Step	Command	Why
1	`python -m venv .venv`	Isolates the pinned dependencies
2	`pip install -r requirements-dev.txt` (or `requirements.txt` to only run the app)	Installs Django, Matplotlib, python-dotenv (+ test/lint tools)
3	copy `.env.example` to `.env`	Sets `DJANGO_DEBUG=true` and a placeholder secret; without a secret and with DEBUG off, settings raise `ImproperlyConfigured`
4	`python manage.py migrate`	Creates the 23 tables
5	`python manage.py seed_codeguard --demo-users`	Stages, controls, six scenarios, and three development-only accounts
6	`python manage.py runserver`	Development server on `http://127.0.0.1:8000/`

`python manage.py createsuperuser` creates a real administrator (superusers are always treated as admin by `get_role`).

1.3 Day-to-day loop

1. Change code; for models, follow the workflow in `DATABASE_AND_MIGRATIONS.md` §2.9.
2. For UI work, follow `DESIGN.md` and use the CSS variables (`CLAUDE.md` says the same); `core/tests/test_design.py` fails on colour literals.
3. Run `pytest` and `ruff check . && ruff format --check .`, or `scripts\ci_local.ps1` for every gate.
4. Templates are cached when the server runs with `--noreload` (`tools/a11y/README.md`).

1.4 Management commands

Command	App	Purpose
`seed_codeguard [--reset-scenarios] [--demo-users]`	threats	Idempotent reference data
`backup_db [--output DIR] [--keep N]`	core	Online, integrity-checked SQLite backup

Command	App	Purpose
`purge_audit_events [--days N] [--dry-run]`	audit	Retention; refuses < 30 days
Django built-ins	—	`migrate`, `makemigrations`, `createsuperuser`, `check --deploy`, `collectstatic`, `showmigrations`, `sqlmigrate`

2. Configuration model

All environment-specific values come from environment variables, optionally from `.env` (full list: environment variables). The single switch `DJANGO_DEBUG=false` changes the defaults of `SECURE_SSL_REDIRECT`, secure cookies and HSTS (3600 s), requires a real secret key, and empties `ALLOWED_HOSTS` unless set. Settings validate their own enumerations at import time (`CODEGUARD_DEFAULT_ROLE`, `CODEGUARD_REGISTRATION`, `CODEGUARD_LOG_FORMAT`), so a typo stops the process at start-up instead of misbehaving.

`codeguard/test_settings.py` is a thin overlay for tests and CI; there is no separate production settings module.

3. Git and GitHub

- Remote: `origin` → `https://github.com/bsclmr111626/software-engineering-project.git`.
- History on `main` (10 commits, 2026-10-27 to 2026-11-10), all authored by `bsclmr134322-collab <bsclmr134322@spu.ac.ke>`, from "Add project skeleton and Django settings" to "Add quick start guide to README".
- Project contribution rules: `CONTRIBUTING.md`; issue templates in `.github/ISSUE_TEMPLATE/` (bug report, feature request); `.github/pull_request_template.md`.
- `.gitattributes` and `.gitignore` are present. **Problem found:** the `.gitignore` rule `reports/` (intended for CI output) also matches `templates/reports/`, so `templates/reports/list.html` and `document.html` were never committed. Fixed in commit `eef6716`: the rule is now `/reports/` and both templates are tracked.
- `CLAUDE.md` is listed in `.gitignore` ("Local assistant instructions (not shared)"); `DESIGN.md` is tracked.

4. Continuous integration

Diagram and stage list: CI/CD.

Pipeline	Defined in	Executed?	Deploys?
Jenkins (declared primary)	`Jenkinsfile` (7 stages, Declarative)	**Never on a Jenkins server.** Commands rehearsed by `scripts/rehearse_jenkinsfile.py` on 2026-10-26 (README: SUCCESS). Groovy parsing, plugins (`junit`, `archiveArtifacts`, `recordCoverage`, `timestamps`), `post` blocks and agents are unverified	no
GitHub Actions (supplementary)	`.github/workflows/ci.yml`	**Yes, passing** since run #7 on commit `8ca8b36`, 2026-11-10, 1 min 5 s. Runs #1–#6 failed: missing Reports templates (fixed in `eef6716`), then Ruff formatting Markdown code blocks (fixed in `8ca8b36`)	no
Local	`scripts/ci_local.ps1`, `scripts/ci_local.sh`	Can be run any time	no

The Jenkinsfile's design choices that matter: `disableConcurrentBuilds`, 30-minute timeout, an isolated CI database under `reports/`, seeding twice to prove idempotency, JUnit XML and coverage artefacts, and security checks moved into Python scripts so the Groovy sandbox needs no script approvals (`tests/test_ci_scripts.py::test_jenkinsfile_uses_scripts_and_no_sandbox_restricted_calls`). The README's "How to run it for real" section gives the steps to install Jenkins (Windows or Docker) and create the job.

There is **no CD**: no stage builds an artefact for release, pushes an image or deploys.

5. Deployment

5.1 What exists

`docs/deployment.md` describes one Linux host running a WSGI server behind a TLS-terminating reverse proxy with SQLite on local disk: the production variable table, a first-deployment command sequence, a systemd unit and an nginx block (both marked "example, not tested here"), health checks, backup/restore, retention and upgrades. Its statement of what was verified is accurate: settings, `check --deploy`, the health check, request IDs, JSON logging and the backup/retention commands are tested; the proxy and process-manager snippets are not.

Requirements the code imposes on any deployment:

- `DJANGO_DEBUG=false`, `DJANGO_SECRET_KEY`, `DJANGO_ALLOWED_HOSTS`; `DJANGO_CSRF_TRUSTED_ORIGINS` when the origin differs; `DJANGO_BEHIND_PROXY=true` only behind a proxy that overwrites `X-Forwarded-Proto`.
- HTTPS, because secure cookies and the HTTPS redirect are on by default.
- Static files: `collectstatic` into `staticfiles/` and serve them from the proxy. CodeGuard does **not** include WhiteNoise or any other static-file middleware, so with DEBUG off Django will not serve `/static/` itself.
- A WSGI server installed separately (`gunicorn`/`waitress`; not in requirements).
- The two Reports templates, which a deployment from Git includes only from commit `eef6716` on.

5.2 Hosting options

Status of every option: **not set up, not tested.**

- **Own Linux server or VM** — the path `docs/deployment.md` was written for.
- **PythonAnywhere (free tier)** — discussed as a possibility only. It provides a managed WSGI setup for Django, a static-files mapping and persistent disk, which fit a single SQLite file. Points to check before relying on it: Python 3.12 availability, whether Matplotlib installs on the free tier, the free tier's outbound-network and CPU limits, setting the environment variables in its WSGI file, and HTTPS on the provided subdomain. None of this has been tried.
- **Containers** — no `Dockerfile` exists.

6. Operations

Need	Mechanism
Liveness/readiness	`GET /healthz/` → `200 {"status":"ok","database":"ok"}` or `503`
Correlating logs	`X-Request-ID` header and `[request_id]` in every log line; `CODEGUARD_LOG_FORMAT=json` for collectors
What to alert on	per `docs/deployment.md` §6: health failures, ERROR lines, bursts of `login_throttled`, `simulation_failed`
Backups	`backup_db` nightly; copy off-host; restore drill periodically
Retention	`purge_audit_events --days 365`

Need	Mechanism
Upgrades	backup → deploy → `check --deploy` → `migrate` → `seed_codeguard` → `collectstatic` → restart

7. Troubleshooting

Symptom	Cause in the code	Fix
`ImproperlyConfigured: DJANGO_SECRET_KEY must be set…`	DEBUG false and no key	create `.env` from `.env.example` or set the variable
`ImproperlyConfigured: CODEGUARD_REGISTRATION=domain needs…`	domain mode without domains	set `CODEGUARD_REGISTRATION_DOMAINS`
`no such table`	migrations not applied	`python manage.py migrate`
Empty scenario library / hub	seed not run	`python manage.py seed_codeguard`
`TemplateDoesNotExist: reports/list.html` (500 on `/reports/`)	checkout older than `eef6716` (templates were git-ignored)	update to `eef6716` or later
Login says "Too many failed sign-in attempts"	throttling (5/user, 20/IP, 15 min)	wait, or adjust the `CODEGUARD_LOGIN_*` variables
Signed out with "This session was signed out from another device"	session epoch changed (Settings → Sessions)	sign in again
Pages load without styles in production	static files not collected or not served	`collectstatic` and a proxy `location /static/`
Redirect loop or "insecure" errors behind a proxy	`X-Forwarded-Proto` not trusted	`DJANGO_BEHIND_PROXY=true`
`database is locked`	long write transactions on SQLite	already mitigated (IMMEDIATE + 20 s); otherwise reduce concurrent writers
`python` not found on this machine	per-user Python not on PATH	`.venv\Scripts\python.exe …`
Charts in the wrong colours	JavaScript disabled or bad `?palette=`	expected fallback to the light palette
Demo users refused	`seed_codeguard --demo-users` with DEBUG off	development only by design

Learning guide

A study path through CodeGuard for someone who knows basic Python and wants to understand both Django and this project well enough to explain, change and defend it. Each stage names the files to read, the idea to take away and a hands-on exercise. Exercises that modify code are for a scratch branch; none of them describe features CodeGuard has.

Commands assume the project root and, on this machine, `.venv\Scripts\python.exe` in place of `python`.

Stage 1 — Run it and look around (1 hour)

Do: follow the README Quick start, sign in as `demo_analyst`, run the seeded "Compromised Developer Account" scenario twice: once with no controls and once with MFA, protected branches and SCA. Open each result tab and the report document.

Take away: the vocabulary of the product: stage, scenario, rule, control, run, inherent vs residual risk, blocked vs exposed. Check them against GLOSSARY.md.

Check yourself: why is the second run's residual risk 3.3 and not 0? (Answer in `SIMULATION_AND_RISK_METHODOLOGY.md` §3: blocked and exposed stages keep the likelihood of the failed attempt.)

Stage 2 — How Django turns a URL into a page (2 hours)

Read: `codeguard/urls.py` → `core/urls.py` → `core/views.py:landing` → `templates/core/landing.html` → `templates/layouts/public.html` → `templates/base.html`.

Concepts:

- *URLconf*: `path("", include("core.urls"))` hands the rest of the path to an app; `app_name` creates the namespace used in `reverse("core:landing")` and `{% url %}`.
- *Function-based view*: a function that takes `request` and returns an `HttpResponse`, here via `render(request, template, context)`.
- *Template inheritance*: `{% extends %}` and `{% block %}`; context processors (`TEMPLATES[...]["context_processors"]`) add variables to every template — `accounts.context_processors.role` and `core.context_processors.shell`.
- *Static files*: `{% static %}` and `STATICFILES_DIRS = [BASE_DIR / "static"]`.

Exercise: open `/healthz/` and read `core.views.healthz`. Why is it decorated with `require_GET` and `never_cache`, and why is it listed in `SECURE_REDIRECT_EXEMPT`?

Stage 3 — Models, the ORM and migrations (3 hours)

Read: `threats/models.py`, then `DATABASE_AND_MIGRATIONS.md` in full.

Concepts: fields and validators; `ForeignKey` with `on_delete` (`CASCADE`, `PROTECT`, `SET_NULL`); `ManyToManyField` with and without `through`; `TextChoices`; `Meta.ordering`, `indexes`, `constraints`; querysets are lazy; `select_related` vs `prefetch_related`.

Exercises:

1. `python manage.py shell`, then:

```
from threats.models import ThreatScenario
s = ThreatScenario.objects.get(slug="malicious-dependency")
[(r.source.key, r.target.key, r.probability, r.kind) for r in s.rules.select_related("source",
"target")]
```

Which edge kinds appear, and which edge makes a cycle?

2. Point `DJANGO_DB_PATH` at a scratch file, run `migrate`, then `migrate accounts 0005` and `migrate accounts` again. Read `accounts/migrations/0006_retire_neutral_surface.py` and explain why it uses `apps.get_model`.
3. Run `python manage.py sqlmigrate threats 0003` and explain the four SQL statements per field.

Stage 4 — Forms and validation in layers (2 hours)

Read: `simulation/forms.py`, `threats/forms.py` (`BasePropagationRuleFormSet.clean`), `simulation/engine.py:validate_input`, the `CheckConstraint`s in `threats/models.py`.

Concept: CodeGuard validates the same rule up to three times: in the form (friendly message), in the engine (`SimulationConfigError`, independent of the web layer), and in the schema (last line of defence). Find the three places that stop a self-loop rule.

Exercise: in a test, post a rule formset whose only rule starts at a stage that is not the scenario's entry stage. Which message appears, and which part of `clean()` produced it?

Stage 5 — The engine (4 hours, the core of the project)

Read in this order: `simulation/domain.py`, `simulation/risk.py`, `simulation/propagation.py`, `simulation/engine.py`, `simulation/recommendations.py`, then `SIMULATION_AND_RISK_METHODOLOGY.md`.

Concepts: frozen dataclasses as value objects; pure functions; breadth-first search with a queue (`collections.deque`); a max-product variant of Dijkstra with `heapq`; deterministic ordering; leave-one-out analysis; why rounding is done with `Decimal` and why threshold comparisons are rounded to six places.

Exercises (no database needed):

```
from simulation.tests.factories import make_input
from simulation.engine import simulate
r = simulate(make_input(likelihood=80, threshold=50))
[(s.key, s.state, s.residual_likelihood, s.residual_risk) for s in r.stages], r.residual_risk
```

1. Every edge in `make_input` is 80 %. Predict the likelihood of each stage before running it (80, 64, 51.2, 40.96, 32.768), then check.
2. Set `threshold=81`. Predict the states.
3. Recompute by hand the worked example in the methodology document, then reproduce it in the shell with `services.build_input` against a seeded scratch database.
4. Read `test_most_likely_path_is_used` and design a graph where the most likely path is longer than the shortest one.

Stage 6 — Services, transactions and persistence (2 hours)

Read: `simulation/services.py` (`run_simulation`, `_execute`, `accessible_runs`), `simulation/serialization.py`, `simulation/models.py`.

Concepts: keeping views thin; `transaction.atomic`; why the run row is created *before* the atomic block (so a failure can still be recorded as `failed`); `bulk_create`; JSON snapshots and `snapshot_version`; reproducibility.

Exercise: read `test_failure_marks_run_failed_without_partial_results`. How does it force a failure, and what does it assert about stage results and events?

Stage 7 — Authentication, authorization and middleware (3 hours)

Read: `accounts/permissions.py`, `accounts/forms.py`, `accounts/throttling.py`, `accounts/signals.py`, `accounts/middleware.py`, `core/middleware.py`, `core/observability.py`, `codeguard/settings.py` (security section), then `SECURITY.md`.

Concepts: decorators that wrap views (`role_required` builds `admin_required` and `analyst_required`); object-level access through scoped querysets; returning 404 instead of 403 to avoid disclosure; Django signals (`post_save`, `user_logged_in`, `user_login_failed`); middleware as a chain of callables; CSP with a script hash; context variables for request IDs.

Exercises:

1. Draw the role matrix from the decorators alone, then compare with `SECURITY.md` §3.1.
2. Explain why the throttle check happens in `clean()` *before* `super().clean()`.
3. Explain the proxy problem with `client_ip` (`SECURITY.md` §1).

Stage 8 — Presentation: templates, charts and the design system (2 hours)

Read: `DESIGN.md`, `static/css/codeguard.css` (token block), `core/theme.py`, `simulation/charts.py`, `simulation/visual.py`, `core/svgcharts.py`, `static/js/app.js`.

Concepts: design tokens as CSS variables; preferences rendered as `data-*` attributes; why the theme script must run before CSS applies, and how the CSP hash is computed from the same constant; Matplotlib's object-oriented API on a server; computing SVG geometry in Python and keeping templates dumb; progressive enhancement.

Exercise: request `/simulations/<id>/chart/stages.png?palette=garbage` and explain the result from `parse_palette`.

Stage 9 — Testing and CI (2 hours)

Read: `pytest.ini`, `conftest.py`, `codeguard/test_settings.py`, one file from each layer (`test_risk.py`, `test_services_and_views.py`, `tests/test_integration.py`, `core/tests/test_design.py`), `Jenkinsfile`, `.github/workflows/ci.yml`, `scripts/rehearse_jenkinsfile.py`, then `TESTING_AND_QUALITY_ASSURANCE.md`.

Exercises:

1. Run `pytest --cov` and find the one uncovered line in `simulation/engine.py` (line 99, reported on 2026-11-10). What would a test for it look like?
2. Check out `8f5d092` in a temporary clone and run the tests there. Explain the 9 failures from `.gitignore` alone, then compare with `eef6716`.
3. Explain the difference between "the Jenkinsfile was rehearsed" and "the pipeline ran on Jenkins".

Stage 10 — Explain it to someone else

Prepare a ten-minute walkthrough that covers: the five stages and why they are ordered; one simulation from form to chart (use simulation-workflow); the formula with the worked example; two security controls with their tests; two limitations you would fix first and how. Good candidates from this documentation: MFA for CodeGuard accounts, a trusted-proxy setting for `client_ip`, an uncertainty range for the model.

Self-test questions (answers are in the linked documents)

1. What makes an edge "blocked" rather than "below threshold"? (methodology §2.2)
2. Why can a report survive deletion of its scenario? (data-flow diagram; `SimulationRun` docstring)
3. Which migration contains Python code, and what happens when you reverse it? (DB §2.6)
4. Who can see a private report, and what response does anyone else get? (security §3.2)
5. Why is `pyplot` not used? (architecture §3)
6. Which controls mitigate rather than prevent? (`domain.CATEGORY_MODES`; seed: `sca`, `runtime-monitoring`, `incident-response`)
7. What does `--strict-markers` protect against? (testing §3)
8. What exactly is unverified about the Jenkins pipeline? (deployment §4)

Glossary

Terms as CodeGuard uses them, with the code location that defines each. General terms are defined only as far as this project needs them.

Term	Meaning in CodeGuard	Defined in
Accessible run	A run the user owns, or a completed run someone shared	`simulation.services.accessible_runs`
Admin / Analyst / Viewer	The three roles. Admin manages shared scenarios, users and sees all audit events; analyst creates private scenarios and runs simulations; viewer reads	`accounts.models.Role`, `accounts.permissions`
Applicable control	A control linked to a scenario through `ScenarioControl`; highlighted on the form and preferred when ranking recommendations	`threats.models.ScenarioControl`
Append-only	Audit events cannot be updated or deleted through the ORM; only the retention command removes them	`audit.models.AppendOnlyQuerySet`
Audit event	A record of a security-relevant action with actor, object, sanitised metadata and IP	`audit.models.AuditEvent`
Below threshold	Edge outcome: base probability was already below the threshold	`propagation.OUTCOME_BELOW_THRESHOLD`
Blocked (by controls)	Edge outcome: the edge would have propagated without preventive controls. A stage with such an incoming edge and no successful one is in state `blocked`	`propagation.OUTCOME_BLOCKED`, `STATE_BLOCKED`
Bypass edge	An edge that skips at least one later stage (e.g. IDE → CI/CD)	`propagation.edge_kind`
Classification / band	low (< 15), moderate (≥ 15), high (≥ 30), critical (≥ 50)	`risk.RISK_BANDS`, `risk.classify`
Combined effectiveness / coverage	$100 \times (1 - \prod(1 - \varepsilon_i/100))$ over the controls of one mode protecting a stage	`risk.combined_effectiveness`
Compromised	Stage state: reached by a propagated edge (or the entry stage)	`propagation.COMPROMISED_STATES`
Contribution	Residual risk removed by one enabled control, measured by re-running without it (leave-one-out)	`recommendations.control_contributions`
Control	A security measure in the catalogue with category, effectiveness, effort, priority, protected stages, basis and limitations	`threats.models.SecurityControl`, `domain.Control`
CSP	Content-Security-Policy response header restricting where scripts, styles, fonts and images may come from	`core.middleware.ContentSecurityPolicyMiddleware`
CSRF	Cross-site request forgery; Django's token check on unsafe requests	`CsrfViewMiddleware`, `CSRF_FAILURE_VIEW`
Detective / recovery control	A control in **mitigate** mode: lowers residual risk of a reached stage, never stops propagation	`domain.CATEGORY_MODES`
Effective probability	Edge probability after the target's preventive coverage: $p \times (1 - P/100)$	`propagation.traverse`
Entry stage	Where the scenario's attack starts; compromised by assumption	`ThreatScenario.initial_stage`
Exposed	Stage state: attempted, not compromised, not blocked	`propagation.STATE_EXPOSED`
Forward edge	An edge to the next position	`propagation.edge_kind`

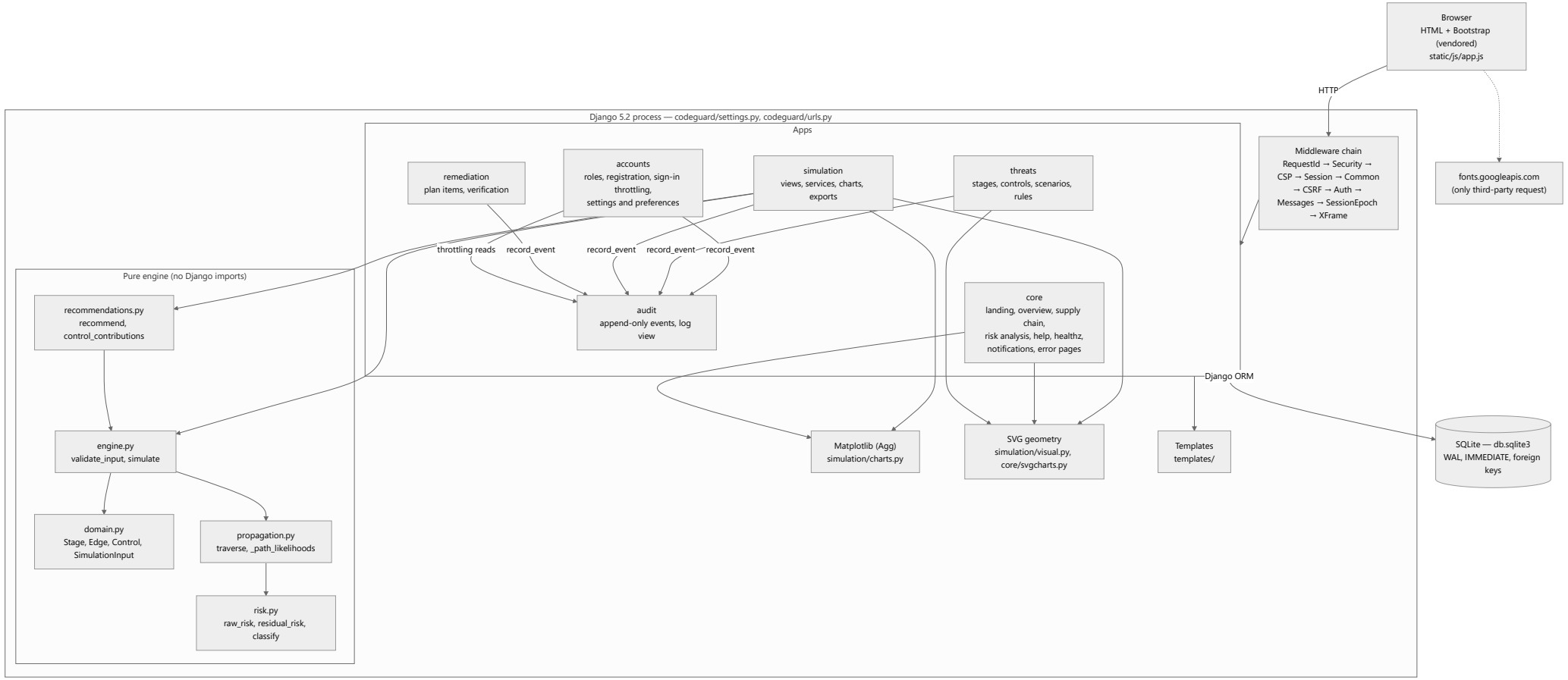
Term	Meaning in CodeGuard	Defined in
IDOR	Insecure direct object reference: reading another user's object by guessing its ID; prevented by scoped querysets returning 404	services modules
Inherent risk	Risk computed with no controls at all	`engine.simulate` (first pass)
Input snapshot	Versioned JSON copy of everything a run used, enough to reproduce it	`simulation.services.scenario_snapshot`
Likelihood (stage)	Probability (0–100) that a stage is compromised: best successful path for compromised stages, strongest failed attempt otherwise	`propagation._path_likelihoods`, `traverse`
Migration	A versioned file of schema/data changes applied by `manage.py migrate`	`*/migrations/`
Mitigating coverage (M)	Combined effectiveness of detective and recovery controls on a stage	`engine._coverage_by_stage`
Model version	`"1.0"`, stored in every snapshot	`engine.MODEL_VERSION`
Overall risk	Weighted average of the five stage risks with stage weights	`risk.weighted_overall`
Preventive control	A control in **prevent** mode: reduces the probability of entering the stages it protects	`domain.MODE_PREVENT`
Preventive coverage (P)	Combined effectiveness of preventive controls on a stage	`engine._coverage_by_stage`
Propagation event	One evaluated edge stored for a run	`simulation.models.PropagationEvent`
Propagation rule	A scenario's directed edge source → target with a probability	`threats.models.PropagationRule`
Propagation threshold (τ)	An edge propagates when its effective probability ≥ τ (default 50)	`ThreatScenario.propagation_threshold`
Raw risk	likelihood × (severity/5) × (impact/5)	`risk.raw_risk`
Recommendation	A disabled control ranked by the modelled residual-risk reduction from re-running with it	`recommendations.recommend`
Remediation item	A recommendation an analyst committed to, with status, due date and note	`remediation.models.RemediationItem`
Request ID	Identifier attached to a request, returned as `X-Request-ID` and written in log lines	`core.observability`
Residual risk	Risk with the enabled controls applied (prevent in propagation, mitigate on the result)	`engine.simulate` (second pass)
Reverse edge	An edge to an earlier position (e.g. Registry → IDE)	`propagation.edge_kind`
Run	One execution of the engine with stored inputs and outputs; status pending, completed or failed	`simulation.models.SimulationRun`
Scenario	A hypothetical attack: entry stage, base likelihood, severity, impact, threshold, rules; shared or private, active or archived, system or custom, versioned	`threats.models.ThreatScenario`
Seed	Idempotent loading of stages, controls and system scenarios	`seed_codeguard`
Session epoch	Counter compared with the session's stored copy; incrementing it signs out other sessions	`UserPreference.session_epoch`, `SessionEpochMiddleware`
Severity / impact	Ordinal 1–5 scenario parameters scaling raw risk	`ThreatScenario`
Shared	For scenarios: visible to every user. For runs: completed and visible read-only to every user	`Visibility.SHARED`, `SimulationRun.is_shared`

Term	Meaning in CodeGuard	Defined in
Stage	One of the five supply-chain nodes, with position, weight and assets	`threats.models.SupplyChainStage`
Supply-chain stages (the five)	IDE / Developer Environment, Version Control, CI/CD Pipeline, Software Registry, Production	`threats/seed_data.STAGES`
System scenario	One of the six seeded scenarios; can be archived, never deleted	`ThreatScenario.is_system`
Throttling	Refusing sign-in after too many recent failures per username or IP	`accounts.throttling`
Transmitted likelihood	likelihood(source) × effective probability / 100 for an evaluated edge	`propagation.traverse`
Unaffected	Stage state: no edge into it was evaluated	`propagation.STATE_UNAFFECTED`
Verification (remediation)	The owner's later completed run of the same scenario with the control enabled	`remediation.services.verification`
WAL	SQLite write-ahead logging, letting readers continue during a write	`DATABASES` options
Weight	Stage criticality (1–5) used in the overall average; seeded 1, 2, 2, 2, 3	`SupplyChainStage.weight`

Appendix A — Diagrams

Diagram: system architecture

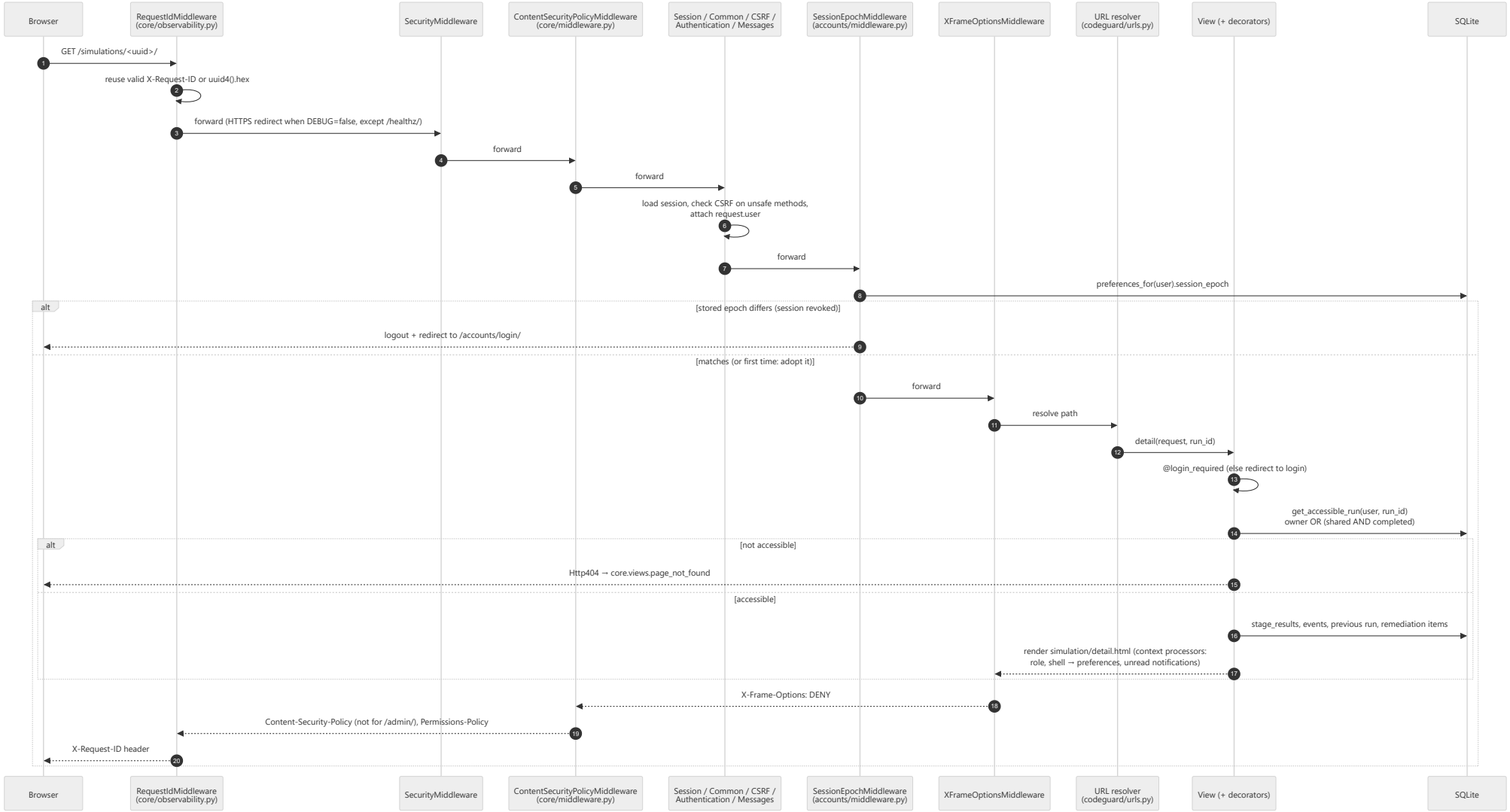
One Django process (any number of identical WSGI workers) serves server-rendered HTML, PNG charts and CSV files from one SQLite file. Arrows show "calls / imports". The simulation engine (`domain`, `engine`, `propagation`, `risk`, `recommendations`) imports nothing from Django.



Source files: `codeguard/settings.py` (`INSTALLED_APPS`, `MIDDLEWARE`, `DATABASES`), `codeguard/urls.py`, `simulation/engine.py` (module docstring: "It never touches the database").

Diagram: request lifecycle

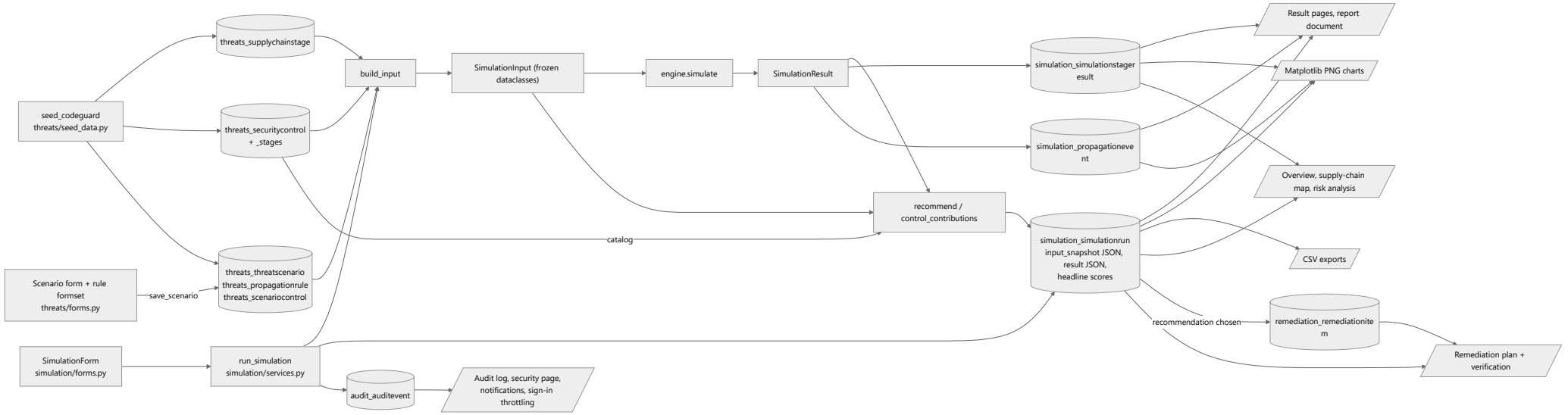
The order of the middleware is the order in `MIDDLEWARE` (`codeguard/settings.py`). On the way out, responses pass back through the same middleware in reverse.



Every log line written during the request carries the ID via `RequestIdFilter` (configured in `LOGGING`).

Diagram: data flow

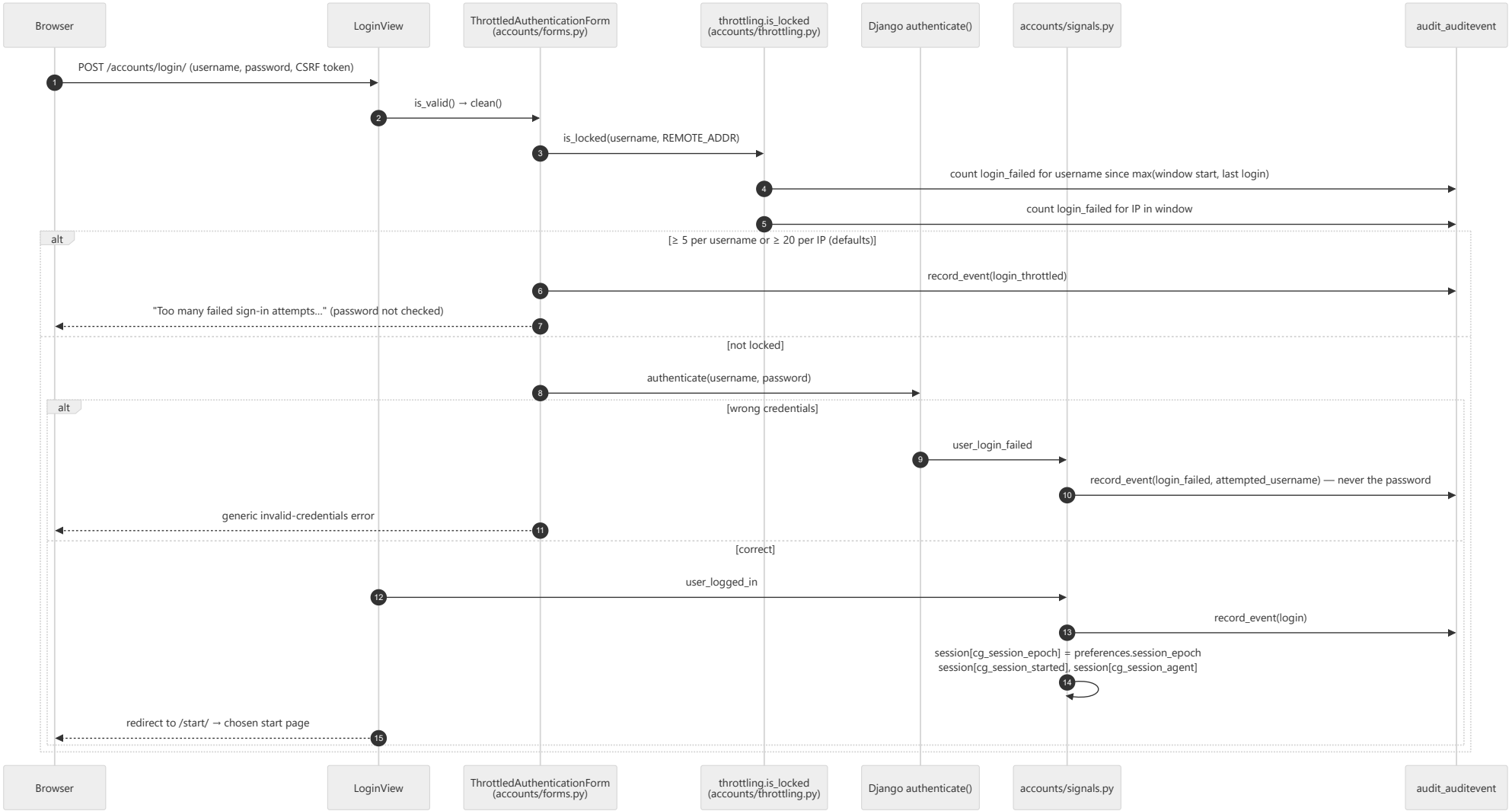
Where data comes from, where it is stored and what reads it. Rectangles are code, cylinders are tables (or JSON columns), parallelograms are outputs.



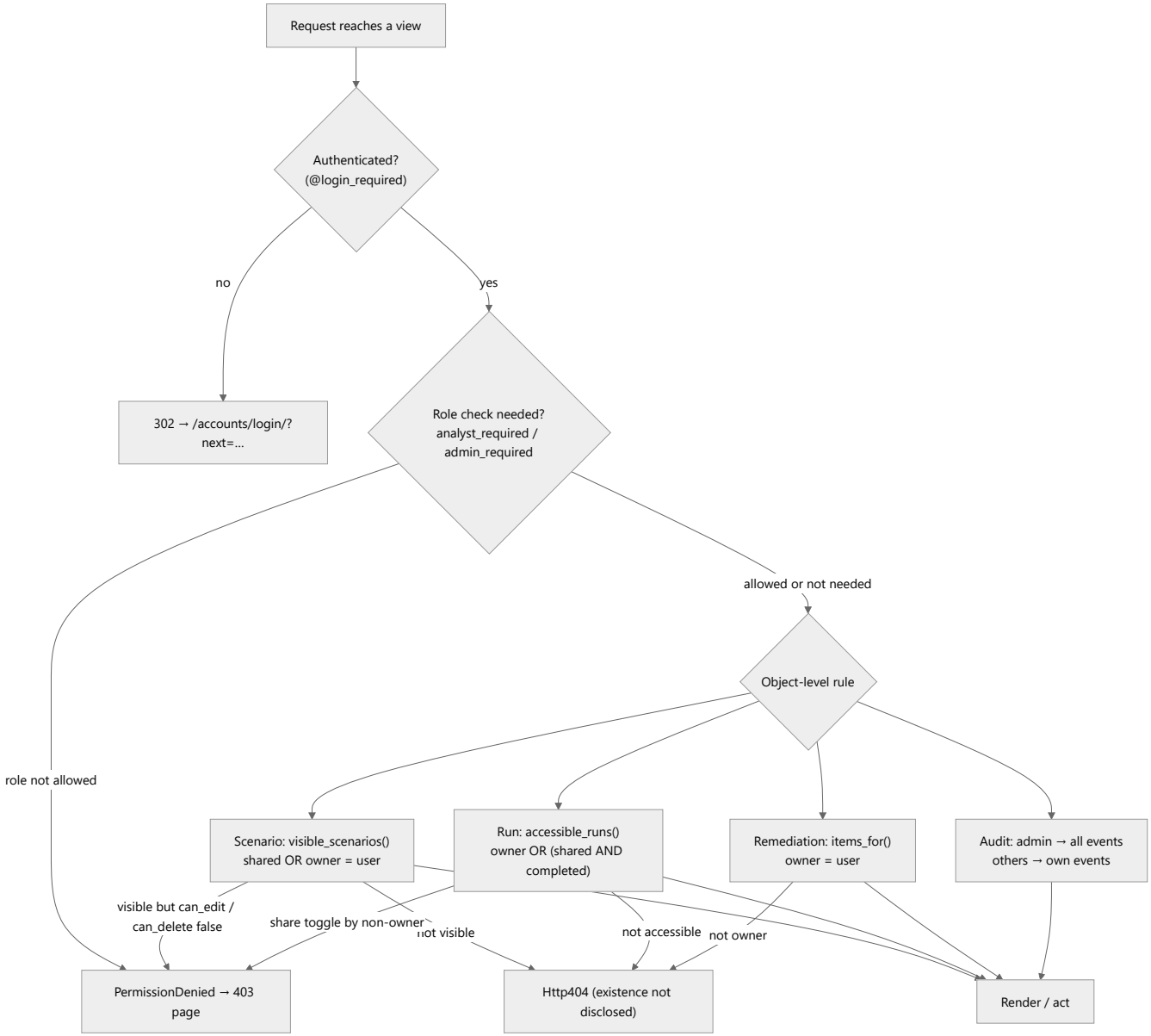
Key property: report pages read only stored rows and JSON (`SimulationRun.input_snapshot`, `result`, stage results, events). Editing or deleting the scenario later does not change them; `simulation.services.input_from_snapshot` can rebuild the exact engine input.

Diagram: authentication and authorization

Sign-in with throttling

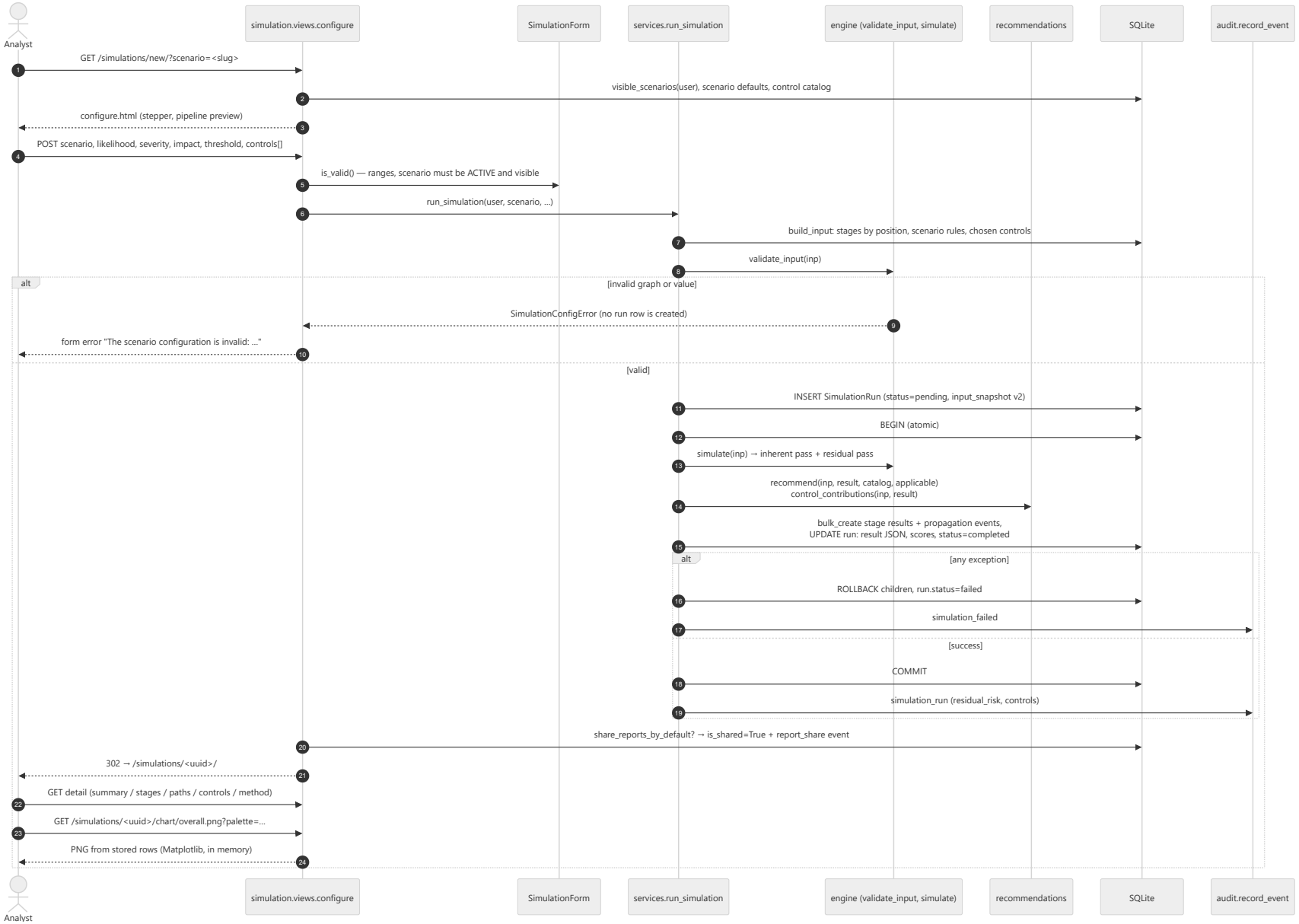


Authorization decision for a protected view



Role resolution (`accounts/permissions.get_role`): anonymous → `None`; superuser → `admin`; otherwise `user.profile.role`, or `viewer` when the profile row is missing.

Diagram: simulation workflow (form to report)



Run states (`simulation.models.RunStatus`):

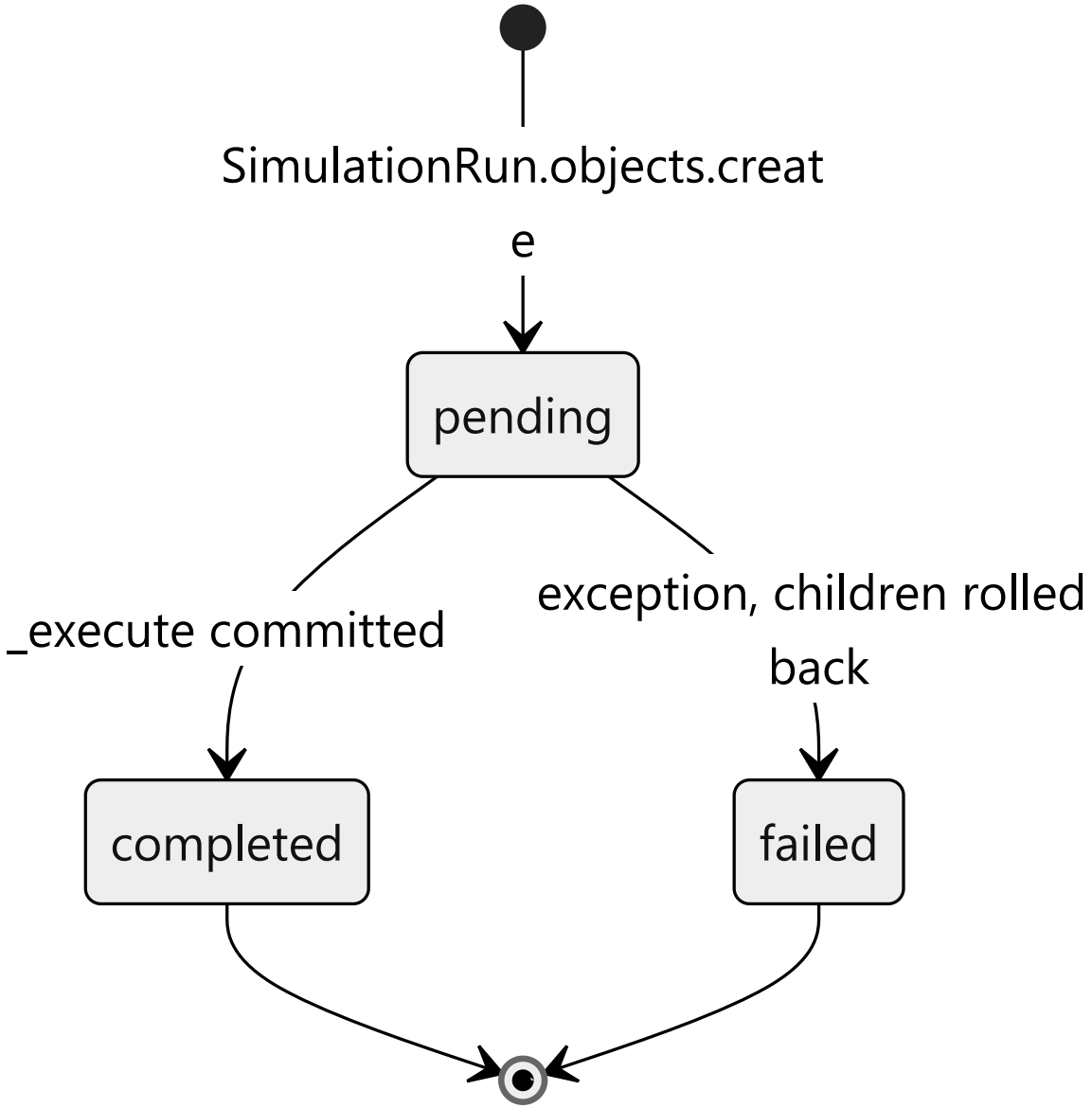
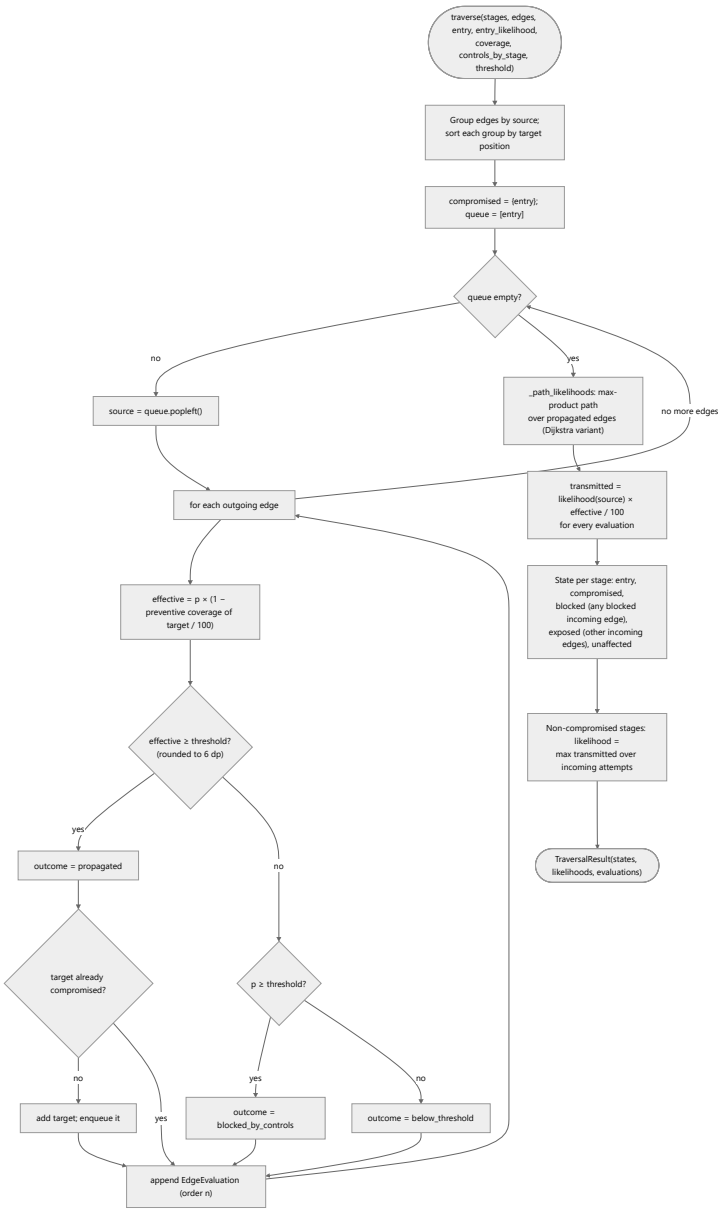


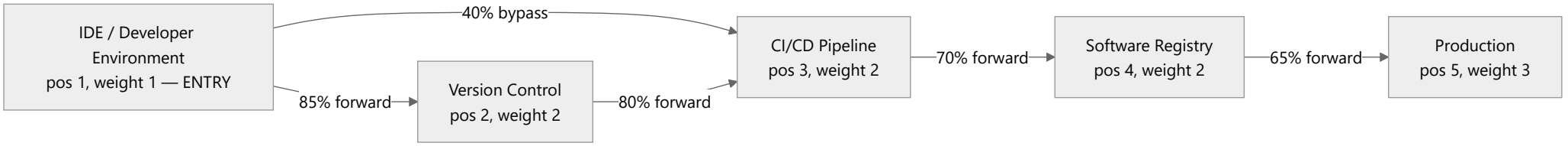
Diagram: propagation algorithm

Activity view of `simulation.propagation.traverse`


traverse(stages, edges, entry, entry_likelihood, coverage, controls_by_stage, threshold)
Group edges by source; sort each group by target position
compromised = {entry}; queue = [entry]
queue empty?
no
yes
source = queue.popleft()
_path_likelihoods: max-product path over propagated edges (Dijkstra variant)
no more edges
for each outgoing edge
transmitted = likelihood(source) × effective / 100 for every evaluation
effective = p × (1 − preventive coverage of target / 100)
State per stage: entry, compromised, blocked (any blocked incoming edge), exposed (other incoming edges), unaffected
effective ≥ threshold? (rounded to 6 dp)
Non-compromised stages: likelihood = max transmitted over incoming attempts
yes
no
outcome = propagated
TraversalResult(states, likelihoods, evaluations)
target already compromised?
p ≥ threshold?
no
yes
no
add target; enqueue it
yes
outcome = blocked_by_controls
outcome = below_threshold
append EdgeEvaluation (order n)

The seeded "Compromised Developer Account" graph

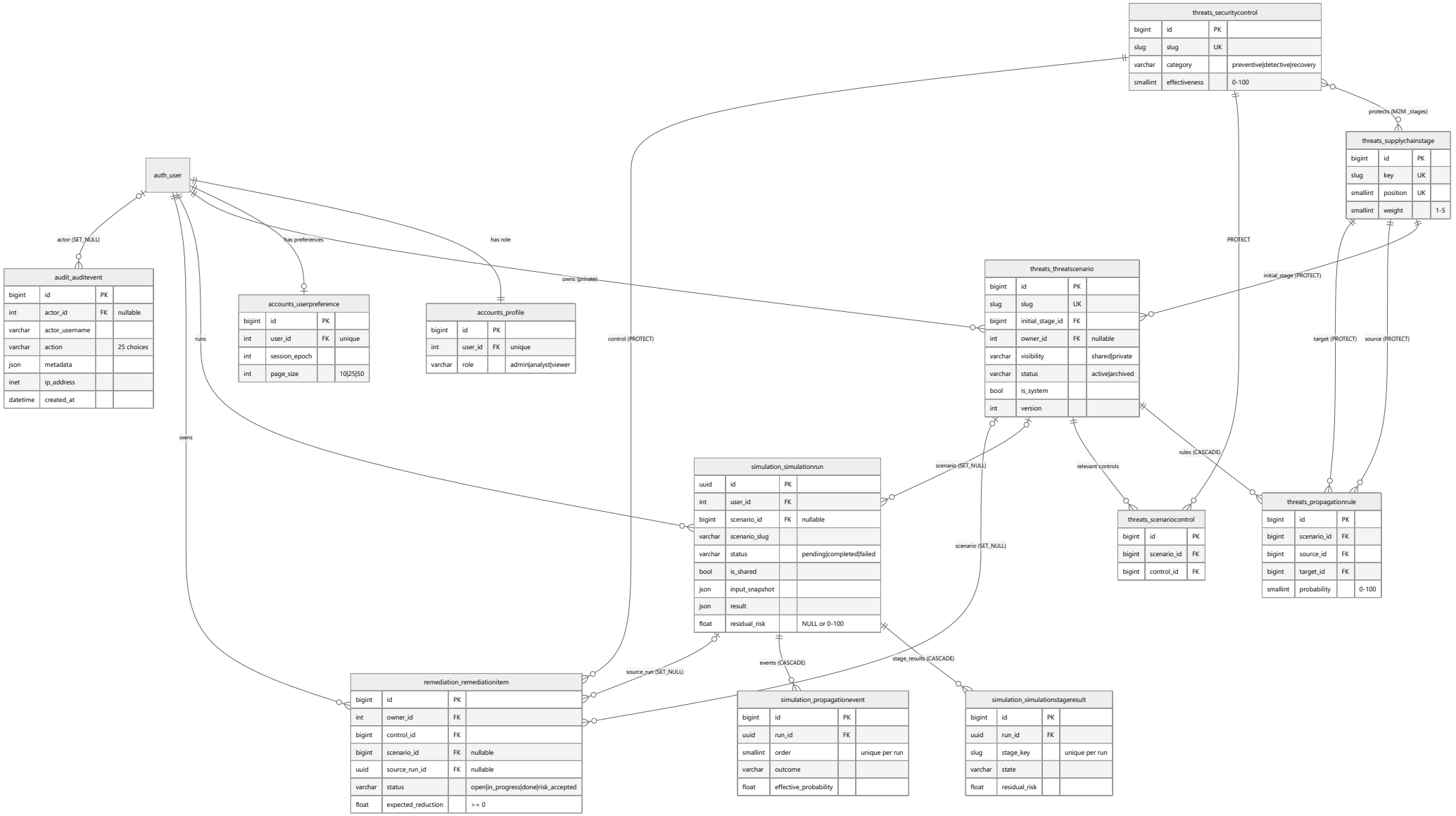
Edge labels are the scenario's propagation probabilities (`threats/seed_data.py`). Edge kinds come from `edge_kind`: next position = forward, later position = bypass, earlier = reverse.



With no controls and threshold 50 %, every edge except `ide → cicd` (40 % < 50 %) propagates and all five stages are compromised. The worked example in `SIMULATION_AND_RISK_METHODOLOGY.md` adds MFA, protected branches and SCA, which block `ide → vcs` and leave only the entry stage compromised.

Diagram: entity-relationship model

Only CodeGuard tables and `auth_user` are shown. Field lists are abbreviated to keys and the fields that matter for relationships and rules; the full list is in `references/models-and-tables.md`.



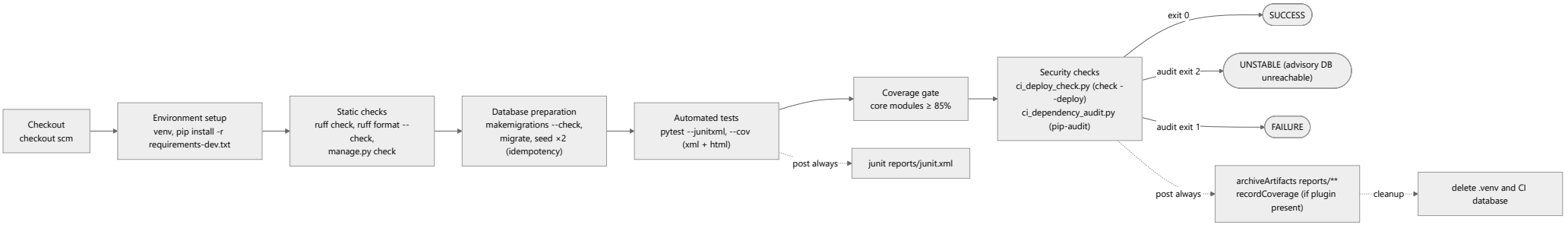
Stage results and events store stage **keys and names as text**, not foreign keys to `threats_supplychainstage`. That is deliberate: a historical report must survive later changes to the stage catalogue.

Diagram: CI/CD

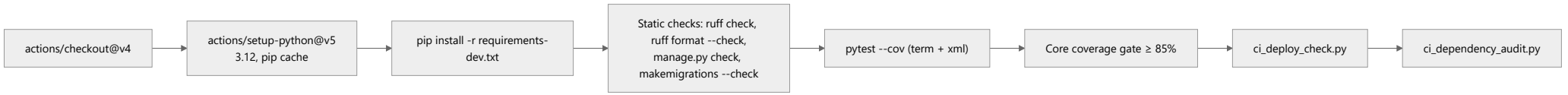
Two pipeline definitions exist. **Neither deploys anything.** Status, as far as can be verified from this machine:

- `Jenkinsfile` — configured, **never executed on a Jenkins server**. Its commands were rehearsed by `scripts/rehearse_jenkinsfile.py` (README: SUCCESS on 2026-10-26).
- `.github/workflows/ci.yml` — configured and **passing**: run #7 on commit `8ca8b36`, 2026-11-10, 1 min 5 s. Runs #1–#6 failed, first because `templates/reports/` was git-ignored (fixed in `eef6716`), then because Ruff's format check rewrote Python quoted in the new Markdown docs (fixed in `8ca8b36`).

Jenkinsfile stages



GitHub Actions job (`test`, ubuntu-latest, Python 3.12)



Triggers: push to `main` and every pull request. Permissions: `contents: read`. Unlike Jenkins it does not run `migrate`/`seed` separately (pytest builds its own database) and treats any non-zero exit from the dependency audit, including the "unreachable" code 2, as a failure.

Local equivalents

`scripts/ci_local.ps1` and `scripts/ci_local.sh` run the same gates on a developer machine; `scripts/rehearse_jenkinsfile.py` clones the repository into a temporary directory and runs every `runCmd` command extracted from the Jenkinsfile, in order.

Appendix B — Reference inventories

Reference: models and database tables

Source of truth: the `models.py` file of each app. Table names were listed from a database built from scratch with `manage.py migrate` on 2026-11-10 (23 tables, below). Django names a table `<app_label>_<lowercase model name>`.

CodeGuard tables (12 models, 13 tables)

Table	Model (file)	Purpose	Key fields and relations	Constraints and indexes
`accounts_profile`	`Profile` (`accounts/models.py`)	The user's CodeGuard role plus optional profile details	`user` one-to-one → `auth_user` (CASCADE); `role` (`admin`/`analyst`/`viewer`, default `viewer`); `job_title`, `team`	`profile_role_valid` CHECK (role in the three values)
`accounts_userpreference`	`UserPreference` (`accounts/models.py`)	Interface, notification and session preferences	`user` one-to-one → `auth_user` (CASCADE); `color_mode`, `accent`, `surface`, `text_size`, `heading_font`, `density`, `timestamp_style`, `page_size`, `start_page`; three accessibility booleans; three `notify_*` booleans; `notifications_seen_at`; `share_reports_by_default`; `session_epoch`	`preference_page_size_valid` CHECK (10, 25, 50)
`audit_auditevent`	`AuditEvent` (`audit/models.py`)	Append-only audit trail	`actor` FK → `auth_user` (SET_NULL, nullable); `actor_username`; `action` (25 choices, indexed); `object_type`, `object_id`, `object_repr`; `metadata` JSON; `ip_address`; `created_at` (indexed)	Index (`actor`, `-created_at`). Append-only enforced in Python (`save`, `delete`, `AppendOnlyQuerySet`), not in SQL
`threats_supplychainstage`	`SupplyChainStage` (`threats/models.py`)	The five ordered stages	`key` (unique slug), `name`, `position` (unique), `description`, `assets`, `icon`, `weight` (1–5)	`stage_weight_range` CHECK
`threats_securitycontrol`	`SecurityControl` (`threats/models.py`)	Control catalogue	`slug` (unique), `name`, `description`, `category` (preventive/detective/recovery), `effectiveness` (0–100), `effort`, `priority`, `addresses`, `effectiveness_basis`, `limitations`	`control_effectiveness_range` CHECK (≤ 100; ≥ 0 comes from the unsigned column)
`threats_securitycontrol_stages`	auto M2M table for `SecurityControl.stages`	Which stages a control protects	`securitycontrol_id`, `supplychainstage_id`	Unique pair (Django default)

Table	Model (file)	Purpose	Key fields and relations	Constraints and indexes
`threats_threatscenario`	`ThreatScenario` (`threats/models.py`)	A hypothetical attack used as simulation input	`slug` (unique), `name`, `description`, `initial_stage` FK → stage (PROTECT); `likelihood`, `severity`, `impact`, `propagation_threshold`; narrative text fields; `owner` FK → `auth_user` (CASCADE, nullable); `visibility` (shared/private); `status` (active/archived); `is_system`; `version`; `controls` M2M through `ScenarioControl`	CHECKs: `scenario_likelihood_range`, `scenario_severity_range`, `scenario_impact_range`, `scenario_threshold_range`, `private_scenario_has_owner`. Index (`status`, `visibility`)
`threats_propagationrule`	`PropagationRule` (`threats/models.py`)	A directed edge of one scenario's graph	`scenario` FK (CASCADE); `source`, `target` FK → stage (PROTECT); `probability` (0–100); `rationale`	`unique_rule_per_edge` UNIQUE (scenario, source, target); `rule_not_self_loop` CHECK; `rule_probability_range` CHECK
`threats_scenariocontrol`	`ScenarioControl` (`threats/models.py`)	Marks a control as relevant to a scenario	`scenario` FK (CASCADE); `control` FK (PROTECT); `note`	`unique_scenario_control` UNIQUE
`simulation_simulationrun`	`SimulationRun` (`simulation/models.py`)	One execution of the engine	`id` UUID primary key; `user` FK (CASCADE); `scenario` FK (SET_NULL, nullable); denormalised `scenario_name`, `scenario_slug`, `scenario_version`, `entry_stage_name`; `status` (pending/completed/failed); `error_message`; `is_shared`; inputs `likelihood`, `severity`, `impact`, `threshold`, `enabled_controls` (JSON list); outputs `inherent_risk`, `residual_risk`, `classification`, `compromised_count`, `blocked_count`; `input_snapshot` JSON; `result` JSON; `created_at`, `completed_at`	`run_residual_risk_range` CHECK (NULL or 0–100). Indexes (`user`, `-created_at`), (`is_shared`, `status`); `status` and `created_at` indexed
`simulation_simulationstageresult`	`SimulationStageResult` (`simulation/models.py`)	Per-stage outcome of a run	`run` FK (CASCADE); `stage_key`, `stage_name`, `position`, `state`, `inherent_state`, likelihoods, coverages, risks, `classification`, `explanation`	`unique_stage_per_run` UNIQUE (run, stage_key); index (`stage_key`, `state`)
`simulation_propagationevent`	`PropagationEvent` (`simulation/models.py`)	One evaluated edge in a run	`run` FK (CASCADE); `order`; source/target key and name; `kind`; `base_probability`, `control_coverage`, `effective_probability`, `threshold`; `outcome`; `target_already_compromised`; `transmitted_likelihood`; `controls_applied` JSON; `explanation` (≤ 300 chars)	`unique_event_order_per_run` UNIQUE (run, order)

Table	Model (file)	Purpose	Key fields and relations	Constraints and indexes
`remediation_remediationitem`	`RemediationItem` (`remediation/models.py`)	A recommended control an analyst committed to act on	`owner` FK (CASCADE); `control` FK (PROTECT); `scenario` FK (SET_NULL); `scenario_slug`, `scenario_name`; `source_run` FK (SET_NULL); `baseline_risk`, `expected_reduction`; `status` (open/in_progress/done/risk_accepted); `note`, `due_date`, `closed_at`	Partial UNIQUE `one_active_item_per_control_and_scenario` (owner, scenario_slug, control) WHERE status in (open, in_progress); `remediation_reduction_non_negative` CHECK; index (`owner`, `status`)

There is one more table than models because `SecurityControl.stages` creates an implicit join table.

Django framework tables (10)

`auth_user`, `auth_group`, `auth_group_permissions`, `auth_permission`, `auth_user_groups`, `auth_user_user_permissions`, `django_admin_log`, `django_content_type`, `django_migrations`, `django_session`. CodeGuard uses the built-in `auth.User`: there is no custom user model. Roles are not Django groups or permissions; they live in `accounts_profile.role`. The `auth_group*` and `auth_*permissions` tables exist only because `django.contrib.auth` and the admin install them.

13 CodeGuard tables plus 10 framework tables give the 23 names in the fresh database:

```
accounts_profile              django_admin_log                 simulation_simulationstageresult
accounts_userpreference       django_content_type              threats_propagationrule
audit_auditevent              django_migrations                threats_scenariocontrol
auth_group                    django_session                   threats_securitycontrol
auth_group_permissions        remediation_remediationitem      threats_securitycontrol_stages
auth_permission               simulation_propagationevent      threats_supplychainstage
auth_user                     simulation_simulationrun         threats_threatscenario
auth_user_groups
auth_user_user_permissions
```

Delete behaviour (on_delete) at a glance

Deleting...	Effect
a user	Their profile, preferences, private scenarios, runs (and the runs' stage results and events) and remediation items are deleted (CASCADE). Their audit events stay; `actor` becomes NULL and `actor_username` keeps the name.
a scenario	Its rules and scenario-control links are deleted. Runs and remediation items keep their denormalised name and slug; their `scenario` link becomes NULL. System scenarios cannot be deleted through the app (`threats.services.can_delete`) or the admin (`ThreatScenarioAdmin.has_delete_permission`).
a stage	Refused while any scenario or rule uses it (PROTECT).
a control	Refused while a scenario link or remediation item uses it (PROTECT).
a run	Its stage results and events are deleted; remediation items keep their data and `source_run` becomes NULL. The app has no "delete run" view.

Reference: URL routes

Source: `codeguard/urls.py` and each included `urls.py`. "Access" is what the view itself enforces: **public** (no login), **login** (`@login_required`), **analyst** (`analyst_required`: admin or analyst), **admin** (`admin_required`). Object-level rules (ownership, sharing, visibility) are listed where they apply. "POST" means the view is decorated `@require_POST`.

Root include order (`codeguard/urls.py`): `""` → core, `accounts/`, `threats/`, `simulations/`, `reports/` (`simulation/report_urls.py`), `settings/` (`accounts/settings_urls.py`), `remediation/`, `audit/`, `admin/`. Custom error handlers: `core.views.bad_request`, `permission_denied`, `page_not_found`, `server_error`; CSRF failures use `core.views.csrf_failure` (setting `CSRF_FAILURE_VIEW`).

core (`core/urls.py`, namespace `core`)

Path	Name	View	Access
`/`	`landing`	`landing`	public (live counts of stages, system scenarios, controls)
`/start/`	`start`	`start`	login; redirects to the user's chosen start page
`/overview/`	`dashboard`	`dashboard`	login
`/overview/risk-distribution.png`	`dashboard_chart`	`dashboard_chart`	login (Matplotlib PNG)
`/dashboard/`	`dashboard_legacy`	`dashboard_redirect`	login; permanent redirect to `/overview/`
`/supply-chain/`	`supply_chain`	`supply_chain`	login
`/supply-chain/<key>/`	`stage_detail`	`stage_detail`	login; 404 for unknown key
`/risk/`	`risk`	`risk`	login; `?scope=all
`/notifications/`	`notifications`	`notifications`	login
`/notifications/read/`	`notifications_read`	`notifications_read`	login, POST
`/help/`	`help`	`methodology`	public (layout switches on login state)
`/healthz/`	`healthz`	`healthz`	public, GET only, `never_cache`; exempt from the HTTPS redirect

accounts (`accounts/urls.py`, namespace `accounts`)

Path	Name	View	Access
`/accounts/register/`	`register`	`views.register`	public; 403 page when `CODEGUARD_REGISTRATION=closed`
`/accounts/login/`	`login`	Django `LoginView` with `ThrottledAuthenticationForm`	public
`/accounts/logout/`	`logout`	Django `LogoutView`	POST (Django 5 requires POST for logout)
`/accounts/password/change/`	`password_change`	`AuditedPasswordChangeView`	login
`/accounts/password/change/done/`	`password_change_done`	Django view	login

Path	Name	View	Access
`/accounts/password/reset/`	`password_reset`	Django `PasswordResetView`	public
`/accounts/password/reset/done/`	`password_reset_done`	Django view	public
`/accounts/reset/<uidb64>/<token>/`	`password_reset_confirm`	Django view	public, token-checked
`/accounts/reset/done/`	`password_reset_complete`	Django view	public
`/accounts/users/`	`user_list`	`RedirectView` → `settings:users`	permanent redirect (legacy link)
`/accounts/users/<id>/role/`	`change_role`	`views.change_role`	admin, POST

settings (`accounts/settings_urls.py`, namespace `settings`)

Path	Name	View (`accounts/settings_views.py` unless noted)	Access
`/settings/`	`index`	`index`	login
`/settings/profile/`	`profile`	`profile`	login
`/settings/account/`	`account`	`account` (email change, needs current password)	login
`/settings/appearance/`	`appearance`	`appearance`	login
`/settings/appearance/quick/`	`quick_theme`	`quick_theme` (header light/dark switch)	login, POST
`/settings/typography/`	`typography`	`typography`	login
`/settings/display/`	`display`	`display`	login
`/settings/accessibility/`	`accessibility`	`accessibility`	login
`/settings/notifications/`	`notifications`	`notifications`	login
`/settings/privacy/`	`privacy`	`privacy`	login
`/settings/privacy/export.json`	`export_data`	`export_data` (own data only)	login
`/settings/security/`	`security`	`security`	login
`/settings/sessions/`	`sessions`	`sessions`	login
`/settings/sessions/revoke/`	`revoke_sessions`	`revoke_sessions`	login, POST
`/settings/organization/`	`organization`	`organization`	admin
`/settings/users/`	`users`	`accounts.views.user_list`	admin
`/settings/users/<id>/role/`	`change_role`	`accounts.views.change_role`	admin, POST

threats (`threats/urls.py`, namespace `threats`)

Path	Name	View	Access
`/threats/`	`list`	`scenario_list`	login; only shared + own private scenarios
`/threats/new/`	`create`	`scenario_create`	analyst
`/threats/controls/`	`controls`	`control_list`	login

Path	Name	View	Access
`/threats/controls/<slug>/`	`control_detail`	`control_detail`	login
`/threats/<slug>/`	`detail`	`scenario_detail`	login; 404 if not visible
`/threats/<slug>/edit/`	`edit`	`scenario_edit`	analyst + `can_edit` (own private; admins also shared)
`/threats/<slug>/archive/`	`archive`	`scenario_archive` (toggles archive/restore)	analyst + `can_edit`, POST
`/threats/<slug>/delete/`	`delete`	`scenario_delete` (GET confirms, POST deletes)	analyst + `can_delete` (never system scenarios)
`/threats/<slug>/clone/`	`clone`	`scenario_clone` (redirects to the copy's edit page)	analyst, POST; scenario must be visible

Ordering matters: `controls/` is declared before `<slug>/`, so a scenario slug of `controls` would be unreachable; `unique_slug()` does not reserve it (minor, not observed in practice).

simulations (`simulation/urls.py`, namespace `simulation`)

Path	Name	View	Access
`/simulations/`	`hub`	`hub`	login
`/simulations/new/`	`configure`	`configure` (`?scenario=`, `?rerun=<run id>`)	analyst
`/simulations/results/`	`results`	`results` (`?scope=all	mine
`/simulations/compare/`	`compare`	`compare` (`?a=&b=`; selection page without them)	login; both runs accessible and completed
`/simulations/compare/chart.png`	`compare_chart`	`compare_chart`	login; same rule
`/simulations/<uuid>/`	`detail`	`detail` (section `summary`)	login; owner or shared+completed
`/simulations/<uuid>/share/`	`share`	`toggle_share`	login, POST, owner only, completed only
`/simulations/<uuid>/<section>/`	`detail_section`	`detail` (`summary`, `stages`, `paths`, `controls`, `method`)	login; same rule; 404 for other sections
`/simulations/<uuid>/chart/<kind>.png`	`chart`	`chart` (`overall`, `stages`, `outcomes`)	login; same rule; completed runs only
`/simulations/<uuid>/export.csv`	`export_run`	`export_run` (audited)	login; same rule

reports (`simulation/report_urls.py`, namespace `reports`)

Path	Name	View (`simulation/views.py`)	Access
`/reports/`	`list`	`history` (`?scope=mine	shared
`/reports/export.csv`	`export_history`	`export_history` (own runs only, audited)	login
`/reports/<uuid>/`	`document`	`report_document` (printable assessment)	login; owner or shared+completed

The Reports pages render `templates/reports/list.html` and `templates/reports/document.html`. These two files were git-ignored until commit `eef6716` (see the master document, "Known gaps").

remediation (`remediation/urls.py`, namespace `remediation`)

Path	Name	View	Access
`/remediation/`	`plan`	`plan` (`?status=`, `?sort=due`	impact
`/remediation/track/`	`track`	`track` (form fields `run`, `control`)	analyst, POST
`/remediation/<id>/`	`detail`	`detail` (GET view, POST update)	login, own items; POST needs analyst/admin

audit (`audit/urls.py`, namespace `audit`)

Path	Name	View	Access
`/audit/`	`log`	`audit_log` (`?action=`, `?user=`, `?view=timeline`	table)

admin

`/admin/` — Django admin site. Superuser / staff only (Django's own check). Not covered by the CodeGuard CSP header (`ContentSecurityPolicyMiddleware` skips paths starting with `/admin/`) and its edits are not written to the CodeGuard audit trail.

Reference: migrations

15 CodeGuard migration files across 5 apps (the `core` app has no models and no migrations). Verified on 2026-11-10: `manage.py makemigrations --check --dry-run` → "No changes detected"; `manage.py migrate` on an empty scratch database applied every migration without error, and `showmigrations` marked all of them `[X]`.

App	Migration	Generated	Depends on	Operations	Kind
accounts	`0001_initial`	2026-10-26	`AUTH_USER_MODEL`	`CreateModel Profile` with `profile_role_valid` CHECK	schema
accounts	`0002_preferences`	2026-10-28	`accounts.0001`, `AUTH_USER_MODEL`	`AddField profile.job_title`, `AddField profile.team`, `CreateModel UserPreference` with `preference_page_size_valid`	schema
accounts	`0003_surface_preference`	2026-10-28	`accounts.0002`	`AddField userpreference.surface` (choices paper, neutral)	schema
accounts	`0004_pearl_surface`	2026-10-28	`accounts.0003`	`AlterField surface` (adds `pearl`)	choices only (no SQL on SQLite)
accounts	`0005_sage_surface`	2026-10-28	`accounts.0004`	`AlterField surface` (adds `sage`)	choices only
accounts	`0006_retire_neutral_surface`	2026-10-28	`accounts.0005`	`RunPython move_neutral_to_paper` (reverse: `noop`), then `AlterField surface` (paper, pearl, sage)	**data** + choices
audit	`0001_initial`	2026-10-26	`AUTH_USER_MODEL`	`CreateModel AuditEvent` (16 actions) with index `audit_audit_actor_i_fb4253_idx`	schema
audit	`0002_new_security_actions`	2026-10-26	`audit.0001`	`AlterField action` (+ `login_throttled`, `audit_purge`)	choices only
audit	`0003_remediation_actions`	2026-10-26	`audit.0002`	`AlterField action` (+ `remediation_create`, `remediation_update`)	choices only
audit	`0004_account_actions`	2026-10-28	`audit.0003`	`AlterField action` (+ `profile_update`, `email_change`, `preferences_update`, `sessions_revoked`, `data_export`; 25 total)	choices only
threats	`0001_initial`	2026-10-26	`AUTH_USER_MODEL`	`CreateModel` for `SecurityControl`, `ScenarioControl`, `SupplyChainStage`, `ThreatScenario`, `PropagationRule`, plus their constraints and indexes	schema
threats	`0002_friendly_rule_constraint_messages`	2026-10-26	`threats.0001`	`AlterConstraint` × 2 (adds `violation_error_message` to `unique_rule_per_edge`, `rule_not_self_loop`)	Python-side message only
threats	`0003_control_assumptions`	2026-10-26	`threats.0002`	`AddField securitycontrol.effectiveness_basis`, `AddField securitycontrol.limitations`	schema (table rebuild on SQLite)
simulation	`0001_initial`	2026-10-26	`threats.0001`, `AUTH_USER_MODEL`	`CreateModel` for `SimulationRun`, `SimulationStageResult`, `PropagationEvent`, constraints and indexes	schema

App	Migration	Generated	Depends on	Operations	Kind
remediation	`0001_initial`	not stated in file header	`simulation.0001`, `threats.0003`, `AUTH_USER_MODEL`	`CreateModel RemediationItem`, partial unique constraint, check constraint, index	schema

Notes:

- The `AuditAction` enum in `audit/models.py` has 25 members and `audit.0004` lists the same 25 choices, which is why `makemigrations --check` finds nothing to do. Choices are part of a field's definition, so every new audit action needs a new `AlterField` migration even though no SQL changes (that is what `audit.0002`–`0004` are).
- Mixed quoting styles (`'…'` in files kept as generated, `"…"` in files reformatted by hand) are cosmetic. Ruff excludes `*/migrations/*` (`pyproject.toml`).
- `remediation.0001` depends on `threats.0003` because `RemediationItem.control` points at `SecurityControl` in its post-`0003` shape.

Reference: dependencies

All Python versions are pinned with `==`. Installed versions in `.venv` were spot-checked on 2026-11-10 (`Django 5.2.18`, `matplotlib 3.11.2`, Python `3.12.10`).

Runtime (`requirements.txt`)

Package	Version	Why CodeGuard needs it	Used in
Django	5.2.18	Web framework: ORM, migrations, templates, auth, sessions, CSRF, admin	everywhere
matplotlib	3.11.2	Server-side PNG charts (Agg backend, object-oriented API)	`simulation/charts.py`
python-dotenv	1.2.4	Loads `.env` into the environment before settings are read	`codeguard/settings.py` (`load_dotenv`)
asgiref	3.13.0	Django dependency (sync/async adapters)	transitive
sqlparse	0.6.0	Django dependency (SQL formatting for `sqlmigrate` etc.)	transitive
tzdata	2026.5	Time-zone database, needed on Windows for `USE_TZ`	transitive
numpy	2.5.4	Matplotlib dependency	transitive
pillow	13.0.0	Matplotlib image backend	transitive
contourpy, cycler, fonttools, kiwisolver, packaging, pyparsing, python-dateutil, six	see file	Matplotlib dependencies	transitive

There is no WSGI server in the requirements. `docs/deployment.md` recommends installing `gunicorn` (Linux) or `waitress` (Windows) separately.

Development and CI (`requirements-dev.txt`, includes `requirements.txt`)

Package	Version	Purpose
pytest	9.1.1	Test runner (`pytest.ini`)
pytest-django	4.14.0	Django integration: `db`, `client` fixtures, temporary test database
pytest-cov	7.1.0	`--cov` option
coverage	7.17.0	Coverage measurement and the `--fail-under=85` gate
ruff	0.17.2	Lint (`E F W I B UP DJ S` rule sets) and formatter
pip-audit	2.10.1	Known-vulnerability scan of `requirements.txt` (`scripts/ci_dependency_audit.py`)

Optional verification tools (not needed to run the app)

Tool	Requirement	Pinned?	Notes
`tools/a11y/*.mjs`	Node.js ≥ 22, Chrome, `axe-core` 4.10.2 (`tools/a11y/package.json`, lockfile committed)	yes	Drives headless Chrome over the DevTools protocol
`tools/authz_live.py`	Python `requests`	**no**	`requests` is importable in this `.venv` (2.34.2, pulled in by `pip-audit`), but no requirements file names it

Frontend assets (vendored, served from `static/`)

Asset	Path	Notes
Bootstrap CSS + bundle JS	`static/vendor/bootstrap/`	Version not recorded in the repository
Bootstrap Icons (CSS + fonts)	`static/vendor/bootstrap-icons/`	Version not recorded
Google Fonts (Instrument Sans, JetBrains Mono, Newsreader)	loaded from `fonts.googleapis.com` in `templates/base.html`	The only third-party request; allowed by the CSP (`style-src`, `font-src`)

Not used

No JavaScript framework or build step, no CDN for JS/CSS, no Celery/Redis, no PostgreSQL driver, no Docker files, no REST framework or JSON API (apart from `/healthz/` and the personal data export).

Reference: environment variables

Read in `codeguard/settings.py` with `os.environ`, after `load_dotenv(BASE_DIR / ".env")`. `python-dotenv` does not override variables that are already set, so real environment variables win over `.env`. Booleans go through `env_bool()`: `1`, `true`, `yes`, `on` (case-insensitive) mean true. Lists go through `env_list()`: comma-separated, blanks dropped.

"In .env.example" says whether the template lists it (commented-out entries count).

Application settings

Variable	Default	Effect	Validated	In .env.example
`DJANGO_DEBUG`	`False`	Debug mode; also flips the production-hardening defaults below	—	yes
`DJANGO_SECRET_KEY`	none	Signing key. If empty: a fixed insecure fallback when DEBUG is true, otherwise `ImproperlyConfigured` at start-up	yes	yes (placeholder)
`DJANGO_ALLOWED_HOSTS`	`localhost,127.0.0.1` when DEBUG, else empty	`ALLOWED_HOSTS`	—	yes
`DJANGO_CSRF_TRUSTED_ORIGINS`	empty	`CSRF_TRUSTED_ORIGINS` (scheme included)	—	yes
`DJANGO_DB_PATH`	`<BASE_DIR>/db.sqlite3`	SQLite file path	—	yes (commented)
`CODEGUARD_DEFAULT_ROLE`	`analyst`	Role of self-registered users	must be `analyst` or `viewer`	yes
`CODEGUARD_REGISTRATION`	`open`	`open`, `domain` or `closed` self-registration	yes; `domain` requires the next variable	yes
`CODEGUARD_REGISTRATION_DOMAINS`	empty	Allowed email domains for `domain` mode (lower-cased)	—	yes (commented)
`CODEGUARD_LOGIN_MAX_FAILURES`	`5`	Failed sign-ins per username in the window before lock	`int()`	yes (commented)
`CODEGUARD_LOGIN_IP_MAX_FAILURES`	`20`	Failed sign-ins per client IP in the window	`int()`	yes (commented)
`CODEGUARD_LOGIN_LOCKOUT_MINUTES`	`15`	Sliding window length	`int()`	yes (commented)
`CODEGUARD_LOG_FORMAT`	`text`	`text` or `json` log lines	yes	yes (commented)

Variable	Default	Effect	Validated	In .env.example
`CODEGUARD_LOG_LEVEL`	`INFO`	Level of the `codeguard` logger	—	yes (commented)

Email (password reset)

Variable	Default	In .env.example
`DJANGO_EMAIL_BACKEND`	`django.core.mail.backends.console.EmailBackend` (prints mail to the console)	yes (commented)
`DJANGO_DEFAULT_FROM_EMAIL`	`codeguard@localhost`	yes (commented)
`DJANGO_EMAIL_HOST`	`localhost`	yes (commented)
`DJANGO_EMAIL_PORT`	`587`	yes (commented)
`DJANGO_EMAIL_HOST_USER`	empty	yes (commented)
`DJANGO_EMAIL_HOST_PASSWORD`	empty	yes (commented; a secret, never commit a value)
`DJANGO_EMAIL_USE_TLS`	`True`	yes (commented)

Transport security (defaults depend on DEBUG)

Variable	Default when DEBUG=false	Default when DEBUG=true	Setting	In .env.example
`DJANGO_SECURE_SSL_REDIRECT`	true	false	`SECURE_SSL_REDIRECT`	yes (commented)
`DJANGO_SECURE_COOKIES`	true	false	`SESSION_COOKIE_SECURE`, `CSRF_COOKIE_SECURE`	yes (commented)
`DJANGO_HSTS_SECONDS`	`3600`	`0`	`SECURE_HSTS_SECONDS`	yes (commented)
`DJANGO_HSTS_INCLUDE_SUBDOMAINS`	false	false	`SECURE_HSTS_INCLUDE_SUBDOMAINS`	**no**
`DJANGO_BEHIND_PROXY`	false	false	when true, `SECURE_PROXY_SSL_HEADER = ("HTTP_X_FORWARDED_PROTO", "https")`	yes (commented)

Test and CI

Variable	Where	Value
`DJANGO_SETTINGS_MODULE`	`pytest.ini`, `Jenkinsfile`, `.github/workflows/ci.yml`, `scripts/ci_local.*`	`codeguard.test_settings`
`DJANGO_DEBUG`, `DJANGO_SECRET_KEY`, `DJANGO_SECURE_SSL_REDIRECT`	`codeguard/test_settings.py` (`os.environ.setdefault`)	`false`, a test-only key, `false`
`DJANGO_DB_PATH`	`Jenkinsfile`	`${WORKSPACE}/reports/ci-db.sqlite3` (pytest itself still uses a temporary DB)
`CORE_COVERAGE_MIN`	`Jenkinsfile`	`85`
`PIP_DISABLE_PIP_VERSION_CHECK`, `PYTHONDONTWRITEBYTECODE`	`Jenkinsfile`	`1`
`DJANGO_DEBUG`, `DJANGO_SECRET_KEY` (random), `DJANGO_ALLOWED_HOSTS=ci.invalid`, `DJANGO_HSTS_SECONDS=31536000`, `DJANGO_SECURE_SSL_REDIRECT`, `DJANGO_SECURE_COOKIES`	`scripts/ci_deploy_check.py` (process-local)	production-like values for `check --deploy`

Verification tools

Variable	Tool	Default
`CODEGUARD_URL`	`tools/authz_live.py`, `tools/a11y/*.mjs`	`http://127.0.0.1:8000`
`DJANGO_DB_PATH`	`tools/authz_live.py` (reads the DB to prove it was not changed)	`<repo>/db.sqlite3`
`DEMO_PASSWORD`	`tools/authz_live.py`	the documented development demo password
`CHROME_PATH`	`tools/a11y/cdp.mjs`	the standard Chrome install path for the OS (`google-chrome` on Linux)
`CODEGUARD_ERROR_URL`	`tools/a11y/audit.mjs`	unset; point it at a second server with DEBUG off to audit real error pages
`CODEGUARD_THEME`	`tools/a11y`	light; `dark` audits the dark theme

The local `.env` file was deliberately not opened while writing this documentation.

Reference: automated tests

Collected with `pytest --collect-only` on 2026-11-10: **337 tests in 15 files**. Parametrised tests count once per parameter.

Results actually obtained on 2026-11-10 (Windows 11, Python 3.12.10)

Run	Command	Result
Working tree	`.venv\Scripts\python.exe -m pytest`	**337 passed** in 36 s
Working tree with coverage	`pytest --cov`	337 passed; **96 %** total branch-aware coverage
Core coverage gate	`coverage report --include=simulation/engine.py,…risk.py,…propagation.py,…recommendations.py --fail-under=85`	**99 %**, exit 0 (engine 99 %, propagation 100 %, recommendations 98 %, risk 97 %)
Fresh `git clone` of HEAD `8f5d092`, same interpreter	`pytest`	**9 failed, 328 passed**: `TemplateDoesNotExist: reports/list.html` / `reports/document.html` (the templates were git-ignored; see the master document, "Known gaps")
Fresh `git clone` of `eef6716` (the fix)	`pytest`	**337 passed**
Lint / format	`ruff check .`, `ruff format --check .`	clean (Markdown excluded from formatting since `8ca8b36`)
Migrations	`makemigrations --check --dry-run`	No changes detected
Deployment check	`python scripts/ci_deploy_check.py`	0 errors; 3 warnings: `codeguard.W001` (open registration), `security.W005`, `security.W021` (HSTS subdomains/preload)
Dependency audit	`python scripts/ci_dependency_audit.py`	No known vulnerabilities, exit 0
GitHub Actions CI	push of `8ca8b36`	**run #7 passed** (all gates, 1 min 5 s); runs #1–#6 failed

Not run on 2026-11-10: the Jenkins rehearsal, the Node browser tools and `tools/authz_live.py` (need a running server). Their last recorded results are in the README's "Last verified results" table, dated 2026-10-26/28.

Inventory by file

File	Tests	What it covers
`simulation/tests/test_risk.py`	44	Formula unit tests: `raw_risk`, `residual_risk`, `combined_effectiveness`, `round_score` (half-up, clamp), `classify` band edges (14.9/15/29.9/30/49.9/50), `weighted_overall`, `meets_threshold` float noise, input validation
`simulation/tests/test_engine.py`	41	`TestPropagation` (chain, threshold inclusive, zero/100 % edges, bypass, reverse, cycles, most-likely path, weighted overall), `TestControls` (preventive blocks, mitigating reduces risk but not propagation, entry-stage prevention, bounded combination), `TestDeterminism`, `TestValidation`

File	Tests	What it covers
`simulation/tests/test_recommendations.py`	6	Exact modeled reduction, ranking, limit, exclusion of enabled/useless controls, leave-one-out contributions, JSON-safe serialisation
`simulation/tests/test_services_and_views.py`	32	`TestRunPersistence` (children saved, failure rollback, snapshot reproduces result, stability after scenario edit/deletion, invalid config creates no run), `TestConfigureView` (prefill, rerun, range checks, viewer denied, no access to others' private scenarios), `TestReportAccess` (IDOR on report/chart/CSV, sharing, palette, CSV formula injection, history scope), `TestComparison`
`threats/tests/test_scenarios.py`	37	CRUD, versioning, graph validation, database constraints, permissions (analyst vs admin vs viewer, system scenarios), control library, seed command idempotency and demo-user guard
`accounts/tests/test_auth.py`	30	Login/logout, registration (role never admin), registration policy (`open/domain/closed`, `W001`), throttling (6 tests), role changes
`accounts/tests/test_settings.py`	32	Every settings page renders, each preference saved and applied, surface migration rules, email change needs password, session revocation, data export scope, admin-only pages
`audit/tests/test_audit.py`	11	Safe context, sensitive-key stripping, truncation, append-only (save, bulk update/delete), survival after user deletion, retention purge, visibility (admin vs own)
`remediation/tests/test_remediation.py`	11	Track only recommended controls, no duplicates (service and DB constraint), privacy, status transitions and `closed_at`, verification by a later run
`core/tests/test_core.py`	27	Security headers and CSP, theme script hash, health check, request IDs, JSON logs, WAL/IMMEDIATE SQLite, error pages without debug details, dashboard metrics from the DB, backup command
`core/tests/test_areas.py`	15	Supply-chain map, stage page, risk analysis, notifications, result sections and report document, SVG chart geometry
`core/tests/test_design.py`	10	`DESIGN.md` rules enforced on CSS/templates/JS: colours only in token blocks, same tokens in both themes, chart palette order matches the server
`core/tests/test_accessibility.py`	7	Error messages linked to inputs, active navigation marked, report page names its diagram/tables/charts
`tests/test_integration.py`	30	Full analyst journey, every main page renders (parametrised), every seeded scenario simulates, `makemigrations --check`, `manage.py check`
`tests/test_ci_scripts.py`	4	`ci_dependency_audit.classify` exit codes; the Jenkinsfile uses scripts and no sandbox-restricted calls

Fixtures

`conftest.py`: `seeded` (runs `seed_codeguard`), `analyst` (alice), `other_analyst` (bob), `viewer` (victor), `admin_user` (ada), and logged-in clients `analyst_client`, `viewer_client`, `admin_client`. `simulation/tests/factories.py` builds engine inputs without the database (`make_input`, `chain_edges`, a five-stage `STAGES` tuple with weights 1, 2, 2, 2, 3).

Configuration

`pytest.ini`: `DJANGO_SETTINGS_MODULE = codeguard.test_settings`, `testpaths` = the six apps plus `tests`, `addopts = -ra --strict-markers`, `filterwarnings = error::RuntimeWarning`. `codeguard/test_settings.py`: MD5 password hasher (speed), in-memory email backend, test hosts. `pyproject.toml [tool.coverage]`: branch coverage over the six apps, omitting migrations, tests, `admin.py` and `apps.py`.